Производительность языка Rust

Work in progress

Об авторах

Юрий Грибов (yugr, the_real_yugr), https://github.com/yugr

- Разработчик и фанат системного ПО (компиляторов, рантаймов, инструментов верификации и т. п.)
- Руководитель компиляторной команды

Захар Акимов, https://github.com/z-inform

- Разработчик general-purpose и AI компиляторов

Особенности языка Rust

- Язык для высокопроизводительного системного программирования
  - Zero cost/overhead abstractions: don’t pay for what you don’t use + no (or minimal) runtime overhead (итераторы, closures, traits, etc.)
  - Возможность низкоуровневого тюнинга (SIMD, inline assembler, компиляторные интринсики типа __builtin_expect, etc.)
- Поддерживает современные парадигмы программирования
  - Процедурное, функциональное и объектно-ориентированное программирование
  - Статический и динамический полиморфизм
  - Etc.
- Язык сокращает количество видов UB за счёт более строгих правил и динамических проверок
  - Особенно memory safety

Производительность Rust: Минусы

- Динамические проверки
- Обязательная инициализация
- Более консервативные библиотечные функции
  - Устойчивость к некорректным компараторам, DoS-стойкое хэширование и т.д.
- Отсутствие -ffast-math
- Допустимость signed integer overflows
- ...

Производительность Rust: Плюсы

- Больше возможностей для alias analysis
  - По сути restrict-by-default
- Агрессивные оптимизации структур
  - Оптимизация порядка полей и niche optimizations
  - Аналог -fipa-struct-reorg в GCC
- Более эффективные стандартные контейнеры
- ...

Цели доклада

https://pxhere.com/ru/photo/870592

- Насколько “дороги” проверки Rust на практике ?
  - В сравнении с Rust без проверок и Hardened C/C++
  - Каковы непосредственные (лишние инструкции) и вторичные (влияние на другие оптимизации, давление на I$/BTB) последствия проверок
- Способен ли от них избавиться Sufficiently Smart (Heroic) Compiler © ?
- И если нет, то как с ними может справиться программист ?
- Есть ли какие-то особенности языка которые наоборот помогают компилятору ?

Мнение создателей языка

- Steven Klabnik :
  - "Generally Rust is the same order of magnitude as C. Sometimes faster, sometimes slower”
  - “Rust prioritizes memory safety above all else. But speed is a close second"

Анализ накладных расходов

- Как оценить накладные расходы от проверок ?
- В Rust *нет опций* для отключения проверок безопасности
- Мы сравним обычный Rust и пропатченный Rust с отключенными проверками
  - Репозиторий с модифицированными версиями компилятора: yugr/rust-private
  - Базовая ветка yugr/baseline основана на теге 1.87.0
    - Для стабильности измерений отключена рандомизация хэштаблиц и усилено выравнивание функций
- А также покажем оверхед от аналогичных проверок в “безопасном” диалекте C++ (Hardened C/C++)

Rust и C++

- Rust и C++ близки по выразительным возможностям:
  - Поддержка нескольких парадигм программирования (процедурной, ООП и функциональной)
  - Статический и динамический полиморфизм
  - Etc.
- Область применения языков совпадает (высокопроизводительное системное программирование)
  - Zero cost/overhead abstractions
  - Возможность низкоуровневого тюнинга
  - Те же backend-оптимизаторы (LLVM, GCC) и технологии оптимизации (PGO, LTO, etc.)

Чего не будет в докладе: Другие аспекты языков

- Эргономичность, выразительность (в частности self-referential data structures), безопасность, etc.
- На эти темы есть множество других докладов !
  - Rust Features that I Want in C++ (CppNow 2022)
  - Rust vs C++ with Steve Klabnik and Herb Sutter (SW Engineering Daily podcast)
  - The existential threat against C++ and where to go from here (NDC 2024)

Чего не будет в докладе: Параллельный код

- Одним из преимуществ Rust считается простота написания корректного параллельного кода
  - Fearless Concurrency with Rust
- Тема требует отдельного рассмотрения и глубокой экспертизы
  - C++ Russia 2027 anyone ?

Чего не будет в докладе: Неидиоматичный код

- Наша цель – “типичный” код на Rust
  - Без SIMD или компиляторных интринсиков, inline assembler, unsafe (в том числе get_unchecked и wrapping_add), restrict, etc.
  - Стандартные библиотеки
- С помощью unsafe, etc. (и большого количества времени) всегда можно добиться идентичной производительности Rust и эквивалентного C++

Чего не будет в докладе: Неидиоматичный код

- "When you compare languages like C, C++ and Rust, which are by-design on equal footing, you can essentially only compare how easy it is to write performant code or how performant idiomatic code is. Modulo optimizer deficiencies, if you implement the exact same datastructures and same algorithms in these languages, they will perform the same" (Nikita Popov, lead-мэйнтэйнер LLVM)
- “For AoT-compiled languages in particular, clang and rustc are both producing LLVM IR and running mostly the same LLVM optimization passes. So given enough work they can always produce essentially the same thing, making the comparisons pointless” (Scott McMurray, Rust compiler team)

Bounds checks

Детальный разбор

Переполнения буфера

- Morris Worm (1988)
- Запись чрезмерно большого объёма данных в переменную программы

```
char local_buf[32];
sprintf(buf, “Message from user: %s”, received_data);
```

- Переполнение стека (stack overflow)
  - Атаки Stack Smashing, Return-to-libc, Return-Oriented Programming
  - Наиболее стандартизованный и технологичный вид атак
- Переполнение кучи (heap overflow)

https://www.rawpixel.com/image/5958324/free-public-domain-cc0-photo

Распространённость buffer overflow уязвимостей

- Лидирующие позиции в рейтинге наиболее опасных уязвимостей
  - Mitre CWE Top 25 2024 (места 2, 6, 20)
- 70% уязвимостей в продуктах Microsoft и Chromium вызваны ошибками памяти
- 40% атак, вызванных ошибками работы с памятью, вызваны buffer overflow
  - Google Project Zero (2024)
- 80% Memory Safety CVE и 46% Memory Safety KEV в 2024
  - 20%+ из них это stack overflow (наиболее опасная уязвимость)
  - Не лучшая метрика (большая часть CVE это уязвимости веб-приложений)

Memory safety в Rust

- Фронтенд генерирует проверки всех индексных доступов
  - Массивы, slices, контейнеры, etc.
  - Проверки могут быть удалены если оптимизатор может доказать безопасность
- Также используются более безопасные алгоритмы в стандартной библиотеке
  - Например сортировка не упадёт при использовании некорректного компаратора

https://www.picpedia.org/highway-signs/s/safety.html

Накладные расходы

- Выполнение сравнения и условного перехода (2-N тактов)
- Дополнительный регистр для хранения длины для сравнения
- Давление на I$ (промахи в кэш) из-за кода проверки и обработчика паники
- Давление на BTB (branch mispredictions)
  - Adding a (predictable) branch to existing code can increase branch mispredictions (Daniel Lemire)
    - 2-4 кратный рост mispredictions из-за *predictable* ветвления
  - Любители LeetCode знают что resize + operator[] работает быстрее чем push_back (push-dominoes)
- Отключение других оптимизаций
  - Прежде всего векторизатора

Оптимизации в компиляторе

- Во многих случаях компилятор способен
  - Гарантировать корректность доступа и удалить проверки
  - Или вынести проверки из цикла (O(N) -> O(1))
- Эти оптимизации делаются на уровне LLVM
  - InstCombine, SimpleLoopUnswitch, LoopVectorize, etc.
  - Также должны оптимизировать Hardened C/C++
- Оптимизатор был сильно улучшен и многие проблемы прошлых лет успешно оптимизируются в текущей версии Rust
  - Например известные примеры Алекса Кладова (matklad)

Успешная оптимизация

Все проверки вынесены в пролог функции

```rust
#[no_mangle]
pub fn foo(a: &[i32], b: &[i32]) -> i32 {
    let mut ans = 0;
    let n = a.len();
    for i in 0..n {
        ans += a[i] * b[i];
    }
    ans
}
```

```
foo:
        .cfi_startproc
        test    rsi, rsi
        je      .LBB0_1   // n == 0 => вернуть 0
        lea     rax, [rsi - 1]
        cmp     rcx, rax
        jbe     .LBB0_6   // b.len() < n => паника
        xor     eax, eax
        xor     ecx, ecx
        .p2align        4
.LBB0_4:                  // Цикл (без проверок!)
        mov     r8d, dword ptr [rdx + 4*rcx]
        imul    r8d, dword ptr [rdi + 4*rcx]
        lea     r9, [rcx + 1]
        add     eax, r8d
        mov     rcx, r9
        cmp     rsi, r9
        jne     .LBB0_4
...
```

Использовались флаги
-Cno-vectorize-loops
-Cno-vectorize-slp
-Cllvm-args=-unroll-allow-remainder=0

Неуспешная оптимизация

Проверки остались в цикле

```rust
#[no_mangle]
pub fn foo(v: &[i32]) -> i32 {
    let mut ans = 0;
    let chunk_len = 4;
    let num_chunks = v.len() / chunk_len;
    for i in 0..num_chunks {
        for j in 0..chunk_len {
            ans += v[i * chunk_len + j];
        }
    }
    ans
}
```

Использовались флаги
-Cno-vectorize-loops
-Cno-vectorize-slp
-Cllvm-args=-unroll-allow-remainder=0

```
bar:
        ...
        .p2align        4
.LBB0_2:        // Цикл (с проверками!)
        lea     rcx, [r8 - 3]
        cmp     rcx, rsi
        jae     .LBB0_7
        lea     rcx, [r8 - 2]
        cmp     rcx, rsi
        jae     .LBB0_7
        lea     rcx, [r8 - 1]
        cmp     rcx, rsi
        jae     .LBB0_7
        cmp     r8, rsi
        jae     .LBB0_6
        add     eax, dword ptr [rdi + 4*r8 - 12]
        add     eax, dword ptr [rdi + 4*r8 - 8]
        add     eax, dword ptr [rdi + 4*r8 - 4]
        add     eax, dword ptr [rdi + 4*r8]
        add     r8, 4
        dec     rdx
        jne     .LBB0_2
```

Проблемы в оптимизаторе: delayed panics

- В Rust RFC 560 было предложено разрешить компилятору комбинировать вместе связанные проверки
  - Если это полезно для оптимизации
  - И поведение программы не изменится (с точностью до сообщения об ошибке)
- Решение о допустимости подобных оптимизаций пока не принято и в случае индексных проверок эта оптимизация сейчас не делается ☹
- Пример: rust #50759

```
use std::fs::{read, write};

fn main() -> Result<(), std::io::Error> {
    let h = read("/tmp/test")?;
    // Каждый доступ проверяется отдельно ☹
    // (достаточно было бы проверить h[15])
    let b = [h[0], h[1], ... h[15]];
    write("/tmp/test", &b)?;
    Ok(())
}
```

Workarounds: Общие соображения

- В циклах используйте слайсы, а не контейнеры
  - Это упрощает работу alias analysis
  - Пример: fschutt/fastblur #3
- Избегайте сложной индексной арифметики
  - На практике всё что сложнее прибавления константы или сложения двух индексных переменных
  - Даже аффинные операции: Taking Advantage of Auto-Vectorization in Rust
- В нетривиальных случаях избегайте полагаться на оптимизатор
  - Его возможности могут (немонотонно) меняться в разных версиях компилятора
  - “Auto-vectorization is not a programming model”
- Эти же советы актуальны и для Hardened C/C++ !

Workarounds

- Удаление проверок
- Сокращение числа проверок
- Снижение стоимости проверок

Workarounds

- **Удаление проверок**
- Сокращение числа проверок
- Снижение стоимости проверок

Workarounds: Unsafe-код

- Наиболее простое и надёжное решение
- Используется внутри контейнеров и итераторов стандартной библиотеки
- Несколько вариантов:
  - Использование указателей в unsafe-блоках
  - Различные варианты get_unchecked для отключения проверки
  - Хинты для компилятора core::hint::unreachable_unchecked и core::hint::assert_unchecked
- Пример: [rust #36072]

```
     /// Return a reference to the element at `index`.
     ///
-    /// Panics if the index is out of bounds.
-    ///
-    /// Unsafe because index must not equal pos.
-    #[inline(always)]
+    /// Unsafe because index must be within the data slice and not equal to pos.
+    #[inline]
     unsafe fn get(&self, index: usize) -> &T {
         debug_assert!(index != self.pos);
-        &self.data[index]
+        debug_assert!(index < self.data.len());
+        self.data.get_unchecked(index)
     }
```

Workarounds: Использование ограниченных типов

- Ограниченность индекса можно выразить с помощью типов
  - Enums или bool
- Пример: rust #102303

```rust
fn foo(array: &[i16; 3],
       i: usize) -> i16 {
    array[i]
}
```

→

```rust
pub enum Enum {
    A, B, C
}

fn foo(array: &[i16; 3],
           i: Enum) -> i16 {
    array[i as usize]
}
```

Workarounds

- Удаление проверок
- **Сокращение числа проверок**
- Снижение стоимости проверок

Workarounds: Ручные asserts

- Позволяет сделать однократную проверку вместо нескольких
- Пример: [jpeg-decoder #167]

```
+     // optimizer hint to eliminate bounds checks within loops
+     assert!(coefficients.len() == 64);
+
      let mut temp = [Wrapping(0); 64];


      // columns
      for i in 0..8 {
          if coefficients[i + 8] == 0
              && coefficients[i + 16] == 0
              && coefficients[i + 24] == 0
              && coefficients[i + 32] == 0
              && coefficients[i + 40] == 0
              && coefficients[i + 48] == 0
              && coefficients[i + 56] == 0
          {
```

Workarounds: Итераторы

- Многие итераторы используют информацию о размере контейнера для сокращения числа проверок

```
pub fn bro(x: &mut [i32], y: &[i32]) {
    let n = x.len();
    for i in 0..n {
        x[i] ^= y[i];
    }
}
```

```
.LBB0_2:
        cmp     rcx, rax
        je      .LBB0_5
        mov     r8d, dword ptr [rdx + 4*rax]
        xor     dword ptr [rdi + 4*rax], r8d
        lea     r8, [rax + 1]
        mov     rax, r8
        cmp     rsi, r8
        jne     .LBB0_2
```

```
pub fn bro(x: &mut [i32], y: &[i32]) {
    x.iter_mut()
        .zip(y.iter())
        .for_each(|(x, y)| *x += *y);
}
```

```
.LBB0_2:
        mov     ecx, dword ptr [rdx + 4*rax]
        add     dword ptr [rdi + 4*rax], ecx
        lea     rcx, [rax + 1]
        mov     rax, rcx
        cmp     rsi, rcx
        jne     .LBB0_2
```

Компиляция с
-Cno-vectorize-loops

Недостатки итераторов

- “Miracle of zero cost abstraction has a limit" (fridsun at Reddit)
- Сильное разбухание и замедление debug-кода
- Компилятор может не заинлайнить длинные цепочки итераторов
- Лямбды в итераторах принимают ссылки на элементы, что может вызывать проблемы с LLVM alias analysis (rust #106539)
- Chained-итераторы (chain, flat_map, flatten, etc.) генерируют неэффективный код при использовании в for-циклах (“external iteration”)
  - Рекомендуется их применять только с .for_each (“internal iteration”)
  - Are iterators even efficient?

Workarounds: Reslicing

- Reslicing, preslicing или subslicing
- Ранее конструирование слайсов заданного размера
- Заставляет компилятор сделать *однократную* проверку при создании слайса
- Пример: [dropbox/rust-brotli]

```
for i in 0..vec.len() {
    black_box(vec[i]);
}
```

```
let slice = &vec[..vec.len()];
for i in 0..vec.len() {
    black_box(slice[i]);
}
```

Workarounds: Slicing

- Выделение смещённых слайсов лучше делать в два шага
- Это позволит компилятору не задумываться о возможном переполнении
- Пример: Optimizing rav1d, an AV1 Decoder in Rust

```
&vec[i..(i + n)]  →  &vec[i..][..n]
```

Workarounds: Порядок индексации

- Данные лучше читать начиная с самого дальнего элемента

```
let a = x[0];
let b = x[1];
let c = x[2];
```

→

```
let c = x[2];
let b = x[1];
let a = x[0];
```

или даже

```
let a = x[0];
let b = x[1];
let c = x[2];
```

→

```
let [a, b, c] = data[..3] else {
    panic!()
};
```

Workarounds

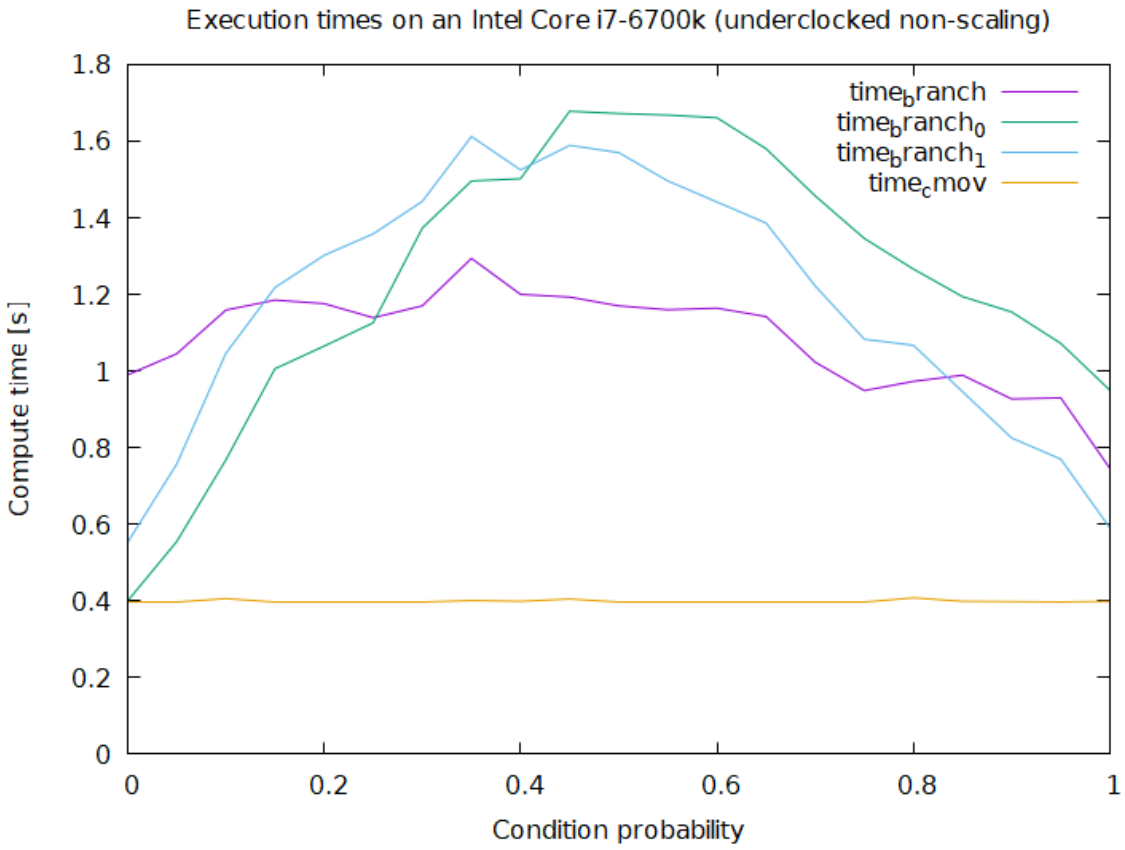
- Удаление проверок
- Сокращение числа проверок
- **Снижение стоимости проверок**

Workarounds: Арифметика вместо проверок

- Проверки можно заменить на инструкцию conditional move

```
let i = cmp::min(i, sums.len());
sums[i] = ...
```

- CMOV *как правило* быстрее чем бранч
- Недостаток: ошибка не будет обнаружена и программа продолжит работу



https://github.com/marcin-osowski/cmov

Workarounds: Арифметика вместо проверок

- Проверки можно заменить на маскирование если длина является 2^N (или может быть дополнена до 2^N)

```
const SIZE: usize = 100;
const LOOKUP_TABLE_SIZE: usize = SIZE.next_power_of_two();

let table: [u64; LOOKUP_TABLE_SIZE] = ...
... table[table & (LOOKUP_TABLE_SIZE - 1)] ...
```

- AND *как правило* может выполняться на большем числе ALU
- К сожалению на данный момент оптимизация применима только для массивов

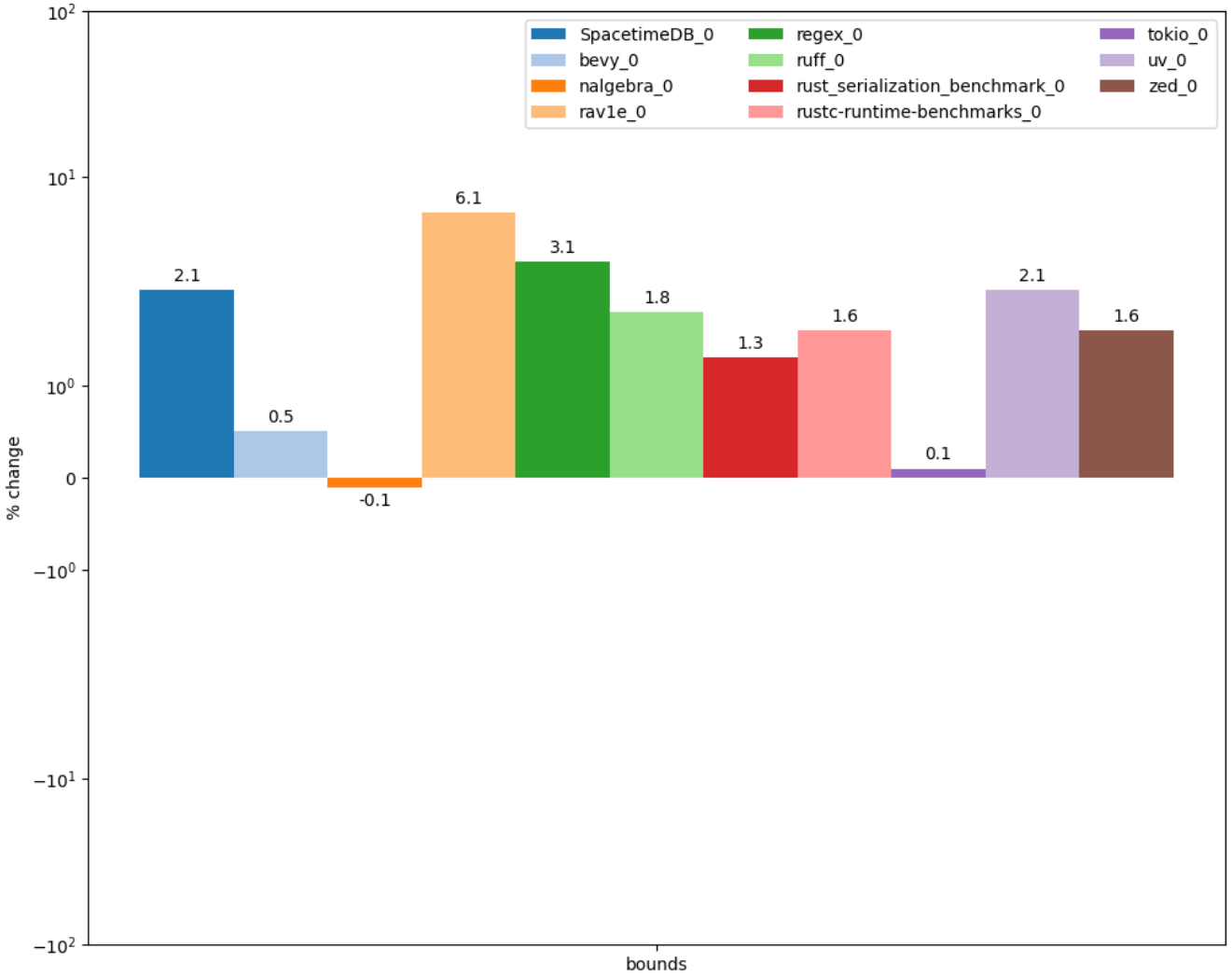
Распространённость

- Более **20%** вызовов panic handlers в librustc_driver.so (ядро компилятора Rust) вызваны индексными проверками
  - Данные получены на основе анализа бинарного кода
- Отключение индексных проверок сокращает число внутренних циклов с panic handlers **в 5 раз**
  - Данные получены с помощью LLVM плагина на основе анализа rustc и oxipng

Методика анализа

Ветка компилятора

Рост производительности при отключении проверок



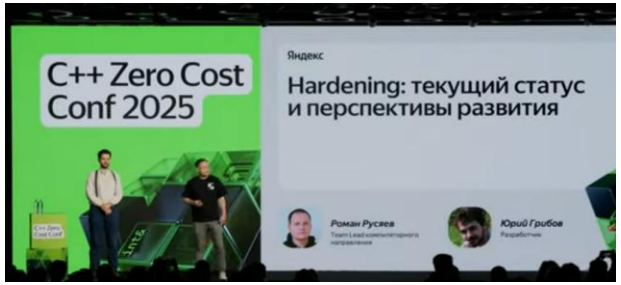
Отключалась инструментация в пользовательском коде и проверки в стандартной библиотеке (контейнеры, slices, strings, etc.)

Ветка компилятора

Hardened C/C++

- В C/C++ доступны ограниченные решения для обеспечения memory safety:
  - StackProtector
  - _FORTIFY_SOURCE
  - Hardened STL (C++ only)
  - -fsanitize=bounds, -fsanitize=object-size
- Полную проверку корректности индексов сделать невозможно из-за отсутствия информации о диапазонах для raw pointers
  - Например только 20% нетривиальных обращений в память в коде Clang могут быть проверены
  - C++ Safe Buffers может быть решением, но требует нетривиальной модификации кода
- Дополнительная информация:
  - Hardening: текущий статус и перспективы развития

Mario Simoes, https://snl.no/Mount_Everest



Rust vs. Hardened C++

```rust
#[no_mangle]
pub fn foo(x: &[i32], i: usize) -> i32 {
    x[i]
}
```

rustc -O -Cpanic=abort

```
foo:
        cmp     rdx, rsi
        jae     .LBB0_2
        mov     eax, dword ptr [rdi + 4*rdx]
        ret
.LBB0_2:
        push    rax
        lea     rax, [rip +
.Lanon.acbd18db4cc2f85cedef654fccc4a4d8.1]
        ...
call    qword ptr [rip +
_ZN4core9panicking18panic_bounds_check@GOTPCREL]
```

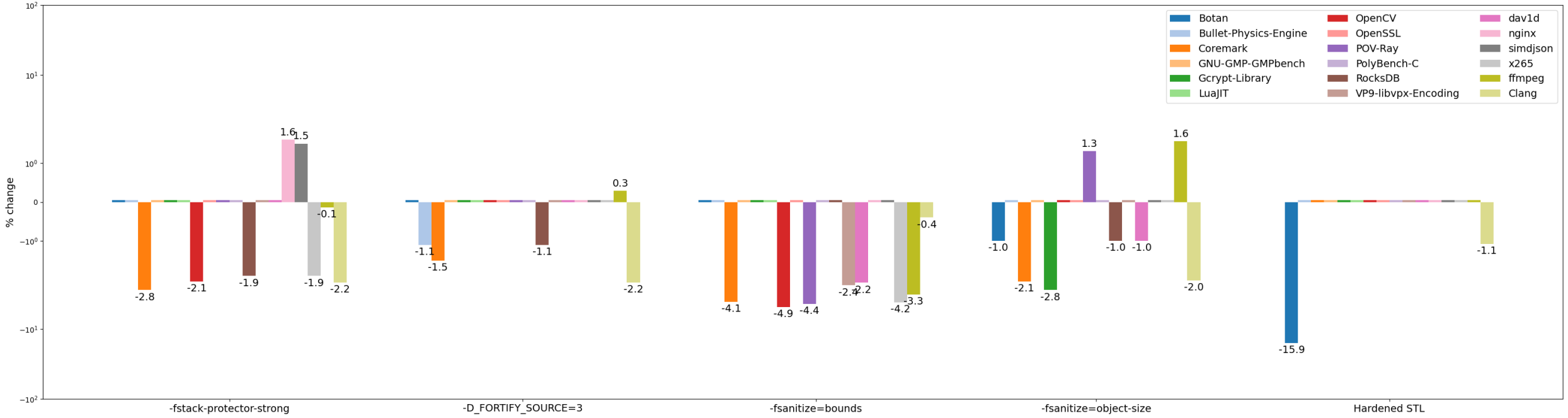
```chinese_word_
#include <span>

extern "C"
int foo(std::span<int> s, size_t i) {
  return s[i];
}
```

gcc -O2 -D_GLIBCXX_ASSERTIONS

```
foo:
.LFB821:
        cmp     rdx, rsi
        jnb     .L7
        mov     eax, DWORD PTR [rdi+rdx*4]
        ret
.L7:
        sub     rsp, 8
        ...
        call
_ZSt21__glibcxx_assert_failPKciS0_S0_@PLT
```

Накладные расходы в C++



Выводы

- Проверки индексов в Rust применяются более агрессивно чем в Hardened C/C++
  - В Hardened C/C++ можно проверить в 5 раз меньше доступов чем в Rust
- Накладные расходы в среднем могут достигать 3-4% (и сравнимы с Hardened C/C++)
  - Но отдельные тесты могут замедляться на десятки процентов (как впрочем и в Hardened C/C++)
- Существует множество способов избежания проверок в критических частях кода
  - Некоторые из них будут не нужны если решат проблему с delayed panics (rust #50759)

Рекомендуемые материалы

- How to avoid bounds checks in Rust without unsafe (Sergey Davidoff aka Shnatsel, Rust Secure WG lead)
  - “Библия” борьбы с индексными проверками
- Story-time: C++, bounds checking, performance, and compilers (Chandler Carruth, SW foundations & PLs technical lead, создатель языка Carbon)

Правила алиасинга

Детальный разбор

Алиасинг указателей

- Универсальная концепция в языках программирования
- Могут ли два разных указателя (ссылки) в программе адресовать одну и ту же память ?
- В разных языках приняты различные подходы:
  - Никакие два указателя в программе не могут ссылаться на одни и те же данные (например Fortran 77)
  - Любые указатели могут ссылаться на одни и те же данные (например C89)
  - Промежуточные варианты
    - Например strict (type-based) aliasing и restrict-аннотации в C99

Почему важен алиасинг ?

- Неограниченный алиасинг удобен для программиста, но снижает возможности компилятора
- Отсутствие алиасинга даёт больше возможностей многим оптимизациям
  - Векторизация, CSE, GVN, LICM, DSE, etc.
- Поэтому важнейшим компонентом любого компилятора является модуль Alias Analysis
  - Идентификация указателей, которые могут (или не могут) алиаситься

```
int alias(int *a, int *b) {
  *a = 0;
  *b = 1;
  return *a;
}

void noalias(int * restrict a, int * restrict b) {
  *a = 0;
  *b = 1;
  return *a;
}
```

```
alias:
        movl    $0, (%rdi)
        movl    $1, (%rsi)
        movl    (%rdi), %eax
        ret

noalias:
        movl    $0, (%rdi)
        xorl    %eax, %eax
        movl    $1, (%rsi)
        ret
```

Теория алиасинга

- Точный межпроцедурный alias analysis неразрешим, а внутри-процедурный NP-полон
  - Undecidability of static analysis (Landi, 1992)
- В реальных программах алиасинг встречается очень редко
  - 98% указателей в бенчмарках SPEC ссылаются только на один объект
    - Dynamic Points-To Sets (Mock et al., 2001)
- Существующие Alias Analysis консервативно переоценивают алиасинг в 1.5-20 раз
  - Dynamic Points-To Sets (Mock et al., 2001)
  - Speculative Alias Analysis for Executable Code (Fernandez et al., 2002)
  - ORAQL — Optimistic Responses to Alias Queries in LLVM (Doerfert et al., 2023)

Алиасинг в Rust

- Правила Borrow checker влияют на производительность:
  - Mutable-ссылка не может алиаситься ни с какими другими (mutable или shared) ссылками
  - Mutable-ссылка, пока она существует, является единственным указателем на объект
  - Объект, адресуемый shared-ссылкой, не может быть изменён (пока она существует)
- На практике это позволяет добиться функциональности, сходной с restrict в C/C++

```
pub fn foo(a: &mut i32,
           b: &mut i32) -> i32 {
    *a = 0;
    *b = 1;
    *a
}
```

```
define i32 @foo(
  ptr noalias ... %a,
  ptr noalias ... %b) {
    store i32 0, ptr %a
    store i32 1, ptr %b
    ret i32 0
}
```

```
foo:
    mov     dword ptr [rdi], 0
    mov     dword ptr [rsi], 1
    xor     eax, eax
    ret
```

Ограничения

- Информация об указателях предоставляется с помощью атрибута noalias у аргументов функций
  - Ссылки, возникающие *внутри* функций, никак не помечаются и с точки зрения LLVM могут алиаситься
- Решение возможно с помощью alias.scope metadata, но пока не реализовано (#16515)
- Workaround: создание dummy-функции для генерации noalias-аннотаций
  - Пример: rustc

```rust
pub fn foo(a: *mut i32,
           b: *mut i32) -> i32 {
    let a = unsafe { &mut *a };
    let b = unsafe { &mut *b };
    *a = 1;
    *b = 2;
    *a
}
```

```asm
foo:
    mov     dword ptr [rdi], 1
    mov     dword ptr [rsi], 2
    mov     eax, dword ptr [rdi]
    ret
```

Ограничения

- Частный случай указанной выше проблемы – алиасинг ссылок на элементы структур данных
- Например при передаче в функцию двух векторов компилятор не сможет определить что их буферы не алиасятся
- Workaround: передавать в функции raw slices

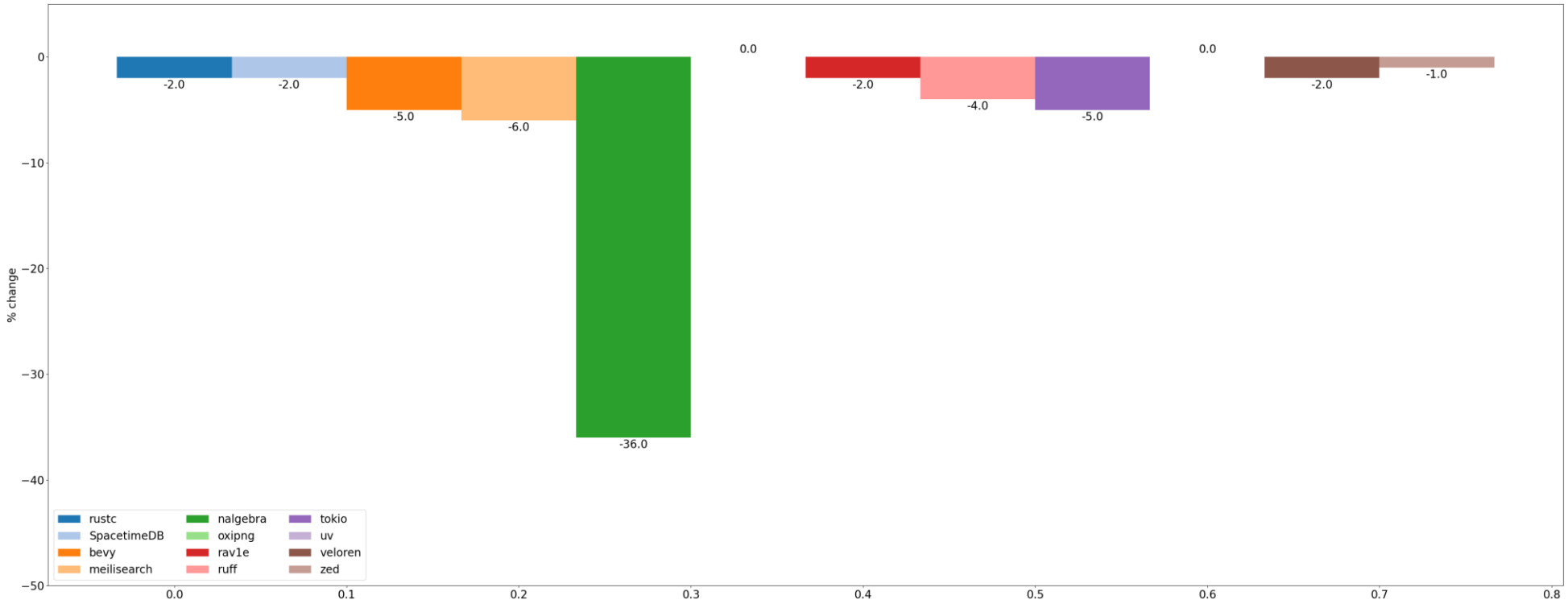
```
pub fn foo(a: &mut Vec<i32>,
           b: &Vec<i32>) {
    let n = a.len();
    let b = &b[..n];
    for i in 0..n {
        a[i] = b[i] + 100;
    }
}
```

```
pub fn foo(a: &mut [i32],
           b: &[i32]) {
    let n = a.len();
    let b = &b[..n];
    for i in 0..n {
        a[i] = b[i] + 100;
    }
}
```

Ограничения

- Правила алиасинга распространяются только на ссылки
- Указатели в Rust будут алиаситься
  - Причём сильнее чем в C/C++ из-за отсутствия strict aliasing

Эффективность правил алиасинга Rust

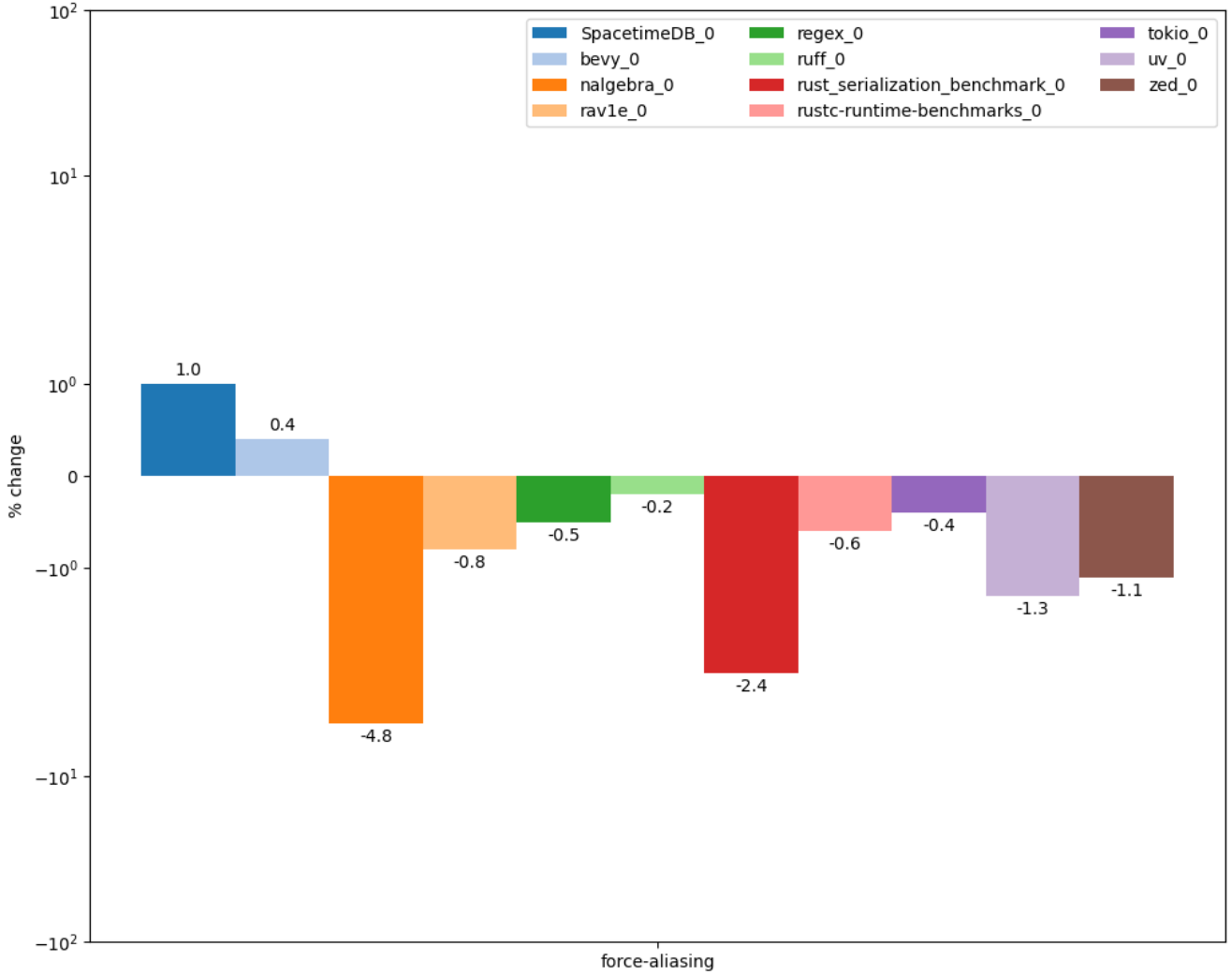


Точность алиас-анализа – число пар указателей, для которых анализ смог дать чёткий ответ (MustAlias / NoAlias)

Ухудшение точности Alias Analysis в Rust при отключении правил алиасинга

Методика анализа Ветка компилятора

Ухудшение производительности при отключении правил алиасинга



Ветка компилятора

Ситуация в C/C++

- C и C++ запрещают алиасинг указателей на разные типы
  - Кроме char *
- Но многие проекты игнорируют это правило (с помощью -fno-strict-aliasing)
- C и C++ предоставляют возможность явного запрета алиасинга программистом с помощью ключевого слова restrict
  - На практике эта функциональность используется редко (в Rust долго не могли включить правила aliasing из-за скрытых багов в LLVM)

Выводы

- Правила алиасинга в Rust позволяют
  - Повысить точность анализа *в среднем* до 5%
  - Улучшить производительность на 1-4%
- После решения проблемы с aliasing metadata в rust #16515 можно ожидать существенного прироста производительности

Рекомендуемые материалы

- О сложности Alias Analysis:
  - Undecidability of static analysis (Landi, 1992)
- Об алиасинге в реальном коде:
  - Dynamic Points-To Sets (Mock et al., 2001)
  - Speculative Alias Analysis for Executable Code (Fernandez et al., 2002)
  - ORAQL — Optimistic Responses to Alias Queries in LLVM (Doerfert et al., 2023)

Обязательная инициализация

Детальный разбор

Пример

```
void foo() {
  char password[32];
  ...
}
void bar() {
  char message[1024];
  if (cond) strcpy(message, "...");
  // Сливаем пароль если !cond
  printf(message);
}
void baz() {
  foo();
  bar();
}
```

Обязательная инициализация

- Неинициализированные переменные приводят к утечкам данных и другим ошибкам
  - Утечки паролей и адресов (утечка адреса компрометирует защиту ASLR)
  - Пример: HeartBleed
- Распространённость:
  - 10% CVE root cause в продуктах Microsoft в 2018 (из Killing Uninitialized Memory)
  - 12% exploitable багов в Android (из P2723)
- Поэтому Rust требует инициализации всех переменных
  - При объявлении или на всех путях программы

Другие языки

- Многие другие языки также требуют инициализацию от программиста (или предоставляют автоинициализацию нулями):
  - C#, Swift, Go, Java
  - Hardened C/C++ (-ftrivial-auto-var-init), C++26 (P2795)
    - Включены по умолчанию в Android user/kernel-space и Chrome
    - Только локальные переменные (глобальные переменные и sbrk/mmap-буферы уже инициализируются, а для кучи нужен hardened allocator)

Компромиссное решение

- Компилятор разрешает не инициализировать переменную, если может доказать что она присваивается на всех путях

```
pub fn foo() {
  let x = 10;
  x
}
```

OK

```
pub fn foo(cond: bool) -> i32 {
  let mut x;
  if cond {
    x = 10;
  } else {
    x = 20;
  }
  x
}
```

OK

```
pub fn foo(cond: u32) -> i32 {
  let mut x;
  if cond > 10 {
    x = 10;
  } else if cond <= 10 {
    x = 20;
  }
  x
}
```

NOT OK

Много ли лишних инициализаций ?

- Сколько ненужных инициализаций приходится делать ?
- Статическая оценка (с помощью LLVM-плагина count-stores):
  - Hardened Clang: +4% stores, 11x вызовов memset (!) в циклах Clang
- Динамическая оценка на примере rustc и clang (с помощью Valgrind-плагина DeadWrites):
  - Компиляция oxipng с помощью rustc:
    - 9% неиспользуемых записей
  - Компиляция LICM.ii с помощью Clang:
    - 11% неиспользуемых записей
  - То же с помощью Hardened Clang (-ftrivial-auto-var-init):
    - 23% неиспользуемых записей (2x !)

Накладные расходы

- Какие последствия имеют лишние инициализации для производительности ?
  - 7% в stdlib file IO (rust #26950)
  - 20-30% в Hyper HTTP (комментарий в tokio #1744)
  - 30% в симуляторе Shadow (shadow #1643)
  - 25% в stdlib TcpStream (RFC #837)
  - 1.5% в rav1d (Making the rav1d Video Decoder 1% Faster)

Накладные расходы

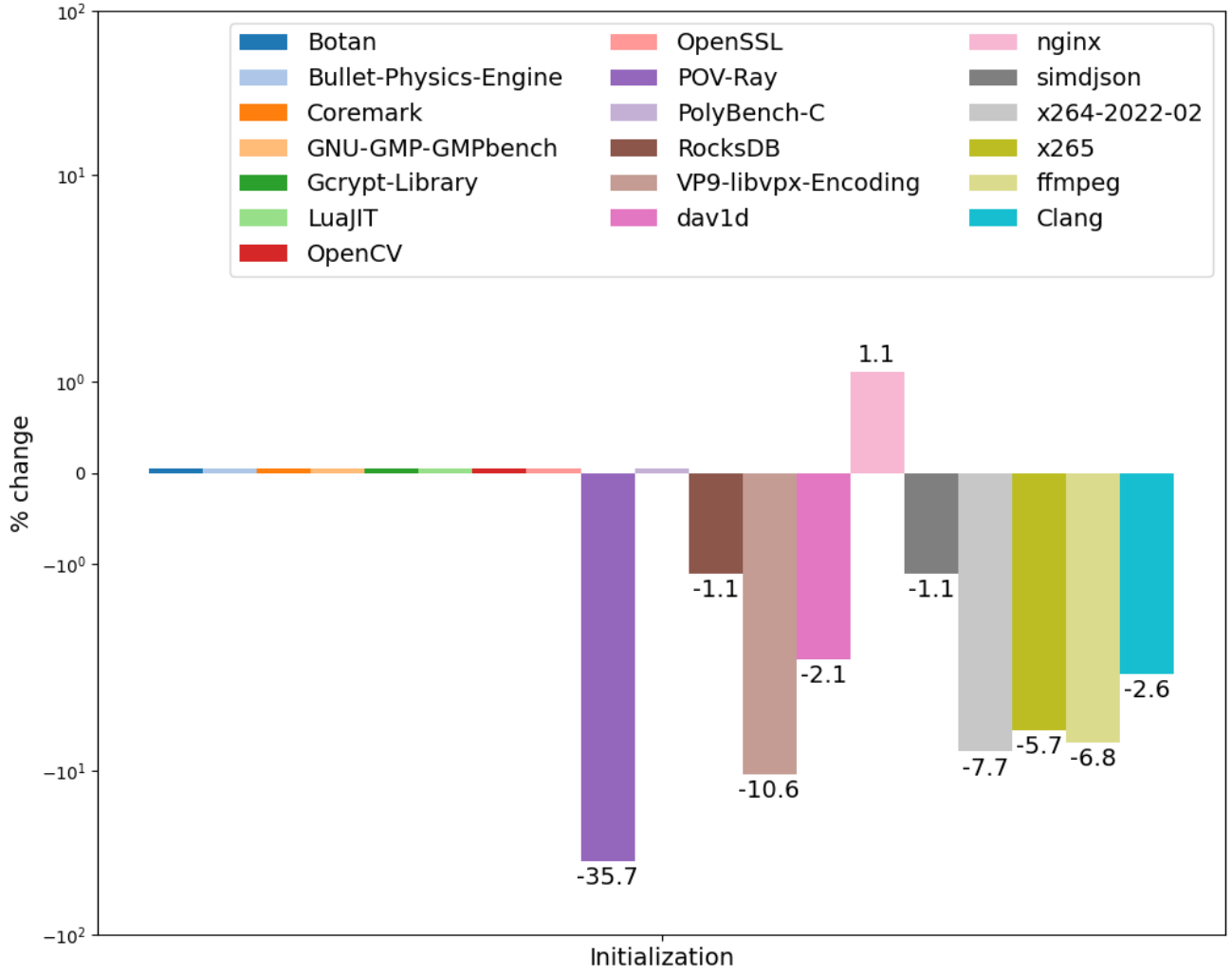
- К сожалению измерить накладные расходы от лишних инициализаций в наших Rust-бенчмарках невозможно
  - Так как потребовалась бы значительная модификация кода
- Проиллюстрируем накладные расходы на примере Hardened C/C++

Накладные расходы (на примере C++)

- Замеры:
  - 1% на Firefox (из Trivial Auto Var Init Experiments)
  - До 10% в горячем коде (virtio, Chrome)
  - 1-3% в среднем на Postgres, но до 20% на некоторых сценариях (Ubuntu #1972043)
  - <1% в Windows (Killing Uninitialized Memory, без ручных оптимизаций 4%)
- Основной проблемный кейс: большой локальный массив (например для IO) на горячем пути

```
while (std::getline(maps, line)) {
      char modulePath[PATH_MAX + 1] =
"";
      // -ftrivial-auto-var-init
      // вставит здесь memset...
      ret = sscanf(line.c_str(),
                   "%lx-%lx %6s %lx %*s
%*x %" PATH_MAX_STRING(PATH_MAX)
                   "s\n",
                   &start, &end, perm,
&offset, modulePath);
    }
```

Накладные расходы при включении компиляторных инициализаций (на примере C++)



Оптимизации в компиляторе

- Инициализации скалярных переменных очень дёшевы и оптимизируются многими SSA-проходами в LLVM
  - DCE, BDCE, ADCE, InstCombine, etc.
- Оптимизации сохранений в память представляют наибольшую проблему
  - DSE (DeadStoreElimination), MoveAutoInit, MemCpyOptimizer
  - В меньшей степени InstCombine

Workarounds

- К сожалению API для работы с неинициализиованной памятью в Rust (MaybeUninit) не очень эргономичны
- Нельзя просто создать ссылку на неинициализированный массив и потом заполнить его:

  “Creating a reference with &/&mut is **only allowed if** the pointer is properly aligned and **points to initialized data**” (addr_of_mut)

Workarounds: Стандартные типы

Массивы:

```
let buf = unsafe {
    let mut buf = MaybeUninit::<[u8;
4]>::uninit();
    let ptr = buf.as_mut_ptr();
    for i in ... {
        ptr.offset(i).write(...);
    }
    buf.assume_init()
};
```

Структуры:

```
let role = unsafe {
    let mut uninit =
MaybeUninit::<Role>::uninit();
    let ptr = uninit.as_mut_ptr();
    // Нужна именно запись
    // (а не присваивание)
    // чтобы не пытаться удалить
    // неинициализированный String
    addr_of_mut!((*ptr).name)
        .write("basic".to_string());
    (*ptr).flag = 1;
    (*ptr).disabled = false;
    uninit.assume_init()
};
```

Workarounds: Векторы

```
let mut buf = Vec<u8>::new();

let old_len = buf.len();
buf.reserve(data.len());

unsafe {
    let ptr = buf.spare_capacity_mut();
    for i in ... {
        ptr[idx].write(...);
    }
    buf.set_len(len + data.len());
}
```

Workarounds: IO

```
let mut buf = [MaybeUninit::uninit(); 4096];
let mut buf = BorrowedBuf::from(&mut buf[..]);
rd.read_buf(buf.unfilled())?;
if buf.len() == 0 {
    break Ok(());
}
// Можно использовать buf.filled()
```

Выводы

- Накладные расходы от дополнительной инициализации локальных переменных в Rust и C++26 сопоставимы
  - Возможно больше в Rust, т.к. там инициализируется и динамическая память
- Эти расходы могут быть достаточно большими в некоторых сценариях (>10%) и требовать ручного тюнинга кода:
  - Использование специальных библиотечных API (типа MaybeUninit и других)

Рекомендуемые материалы

- Killing Uninitialized Memory (video)
  - Доклад от Microsoft о внедрении аналогичных проверок в Visual Studio
- P2795: Erroneous behavior for uninitialized reads
  - To be integrated to C++26

Локализация символов

Детальный разбор

Локализация символов

- Функции, объявленные в единице трансляции (крейт в Rust, cpp-файл в C++), могут быть локальными для неё или глобальными
  - В C/C++ контролируется с помощью static или anonymous namespaces
  - В Rust с помощью pub(crate) или pub(super)
- В Rust и C++ выбраны противоположные решения о дефолтном поведении:
  - В Rust функции по умолчанию локальны для своего модуля (top-level функции – для своего крейта)
  - pub(crate)-функции видимы в пределах своего крейта
  - pub-функции видны в пределах всей программы

Локализация в Rust

```rust
#[inline(never)]  // Simulate inliner fail
pub(crate) fn foo1(x: i32) {
    println!("Hello world {}", x);
}

mod mymod {
    #[inline(never)]  // Simulate inliner fail
    pub(super) fn foo2(x: i32) {
        println!("Goodbye world {}", x);
    }

    #[inline(never)]  // Simulate inliner fail
    pub fn foo3(x: i32) {
        println!("Nice world {}", x);
    }
}

pub use mymod::foo3;

pub fn bar() {
    foo1(1);
    mymod::foo2(2);
    mymod::foo3(3);
}
```

```
$ rustc +baseline test.rs -O --emit=asm
--crate-type=rlib -o- \
    | rustfilt \
    | grep globl
        .globl  test::mymod::foo3
        .globl  test::bar
```

Экспортируются
только foo3 и bar !

Оптимизации в компиляторе

- Локальные функции лучше оптимизируются компилятором
- Как правило такие оптимизации связаны с изменением API или ABI:
  - Межпроцедурное распространение констант (IPSCCP)
  - Замена передачи по ссылке на передачу по значению (ArgumentPromotion)
  - Удаление мёртвых аргументов или возвращаемых значений (DeadArgumentElimination)
  - Релаксация calling convention (замена на coldcc в Transforms/IPO/GlobalOpt.cpp)
  - Межпроцедурная аллокация регистров (IPRA)
- А также
  - Более агрессивный inlining (например всегда инлайнятся однократно вызываемые локальные функции)
  - Улучшенное обнаружение мёртвого кода

Оптимизации в линкере

- В случае динамических библиотек локальные функции
  - Могут вызываться напрямую (минуя PLT/GOT)
  - Не должны разрешаться динамическим загрузчиком
- Детали в докладе Динамические библиотеки и способы их ускорения

Пример оптимизации

```
#[inline(never)]  // Simulate inliner fail
fn foo(b: bool, seed: i32, s: &[i32]) -> i32 {
    let mut ans = seed;
    for &x in s {
        ans += if b { 1 } else { ans ^ x }
    }
    ans
}

pub fn bar(n: i32, s: &[i32]) -> i32 {
    let mut ans = 0;
    for i in 0..n {
        ans += foo(true, i, s);
    }
    ans
}
```

```
foo:
        lea     eax, [rsi + rdi]
        ret
```

Цикл в foo был полностью оптимизирован

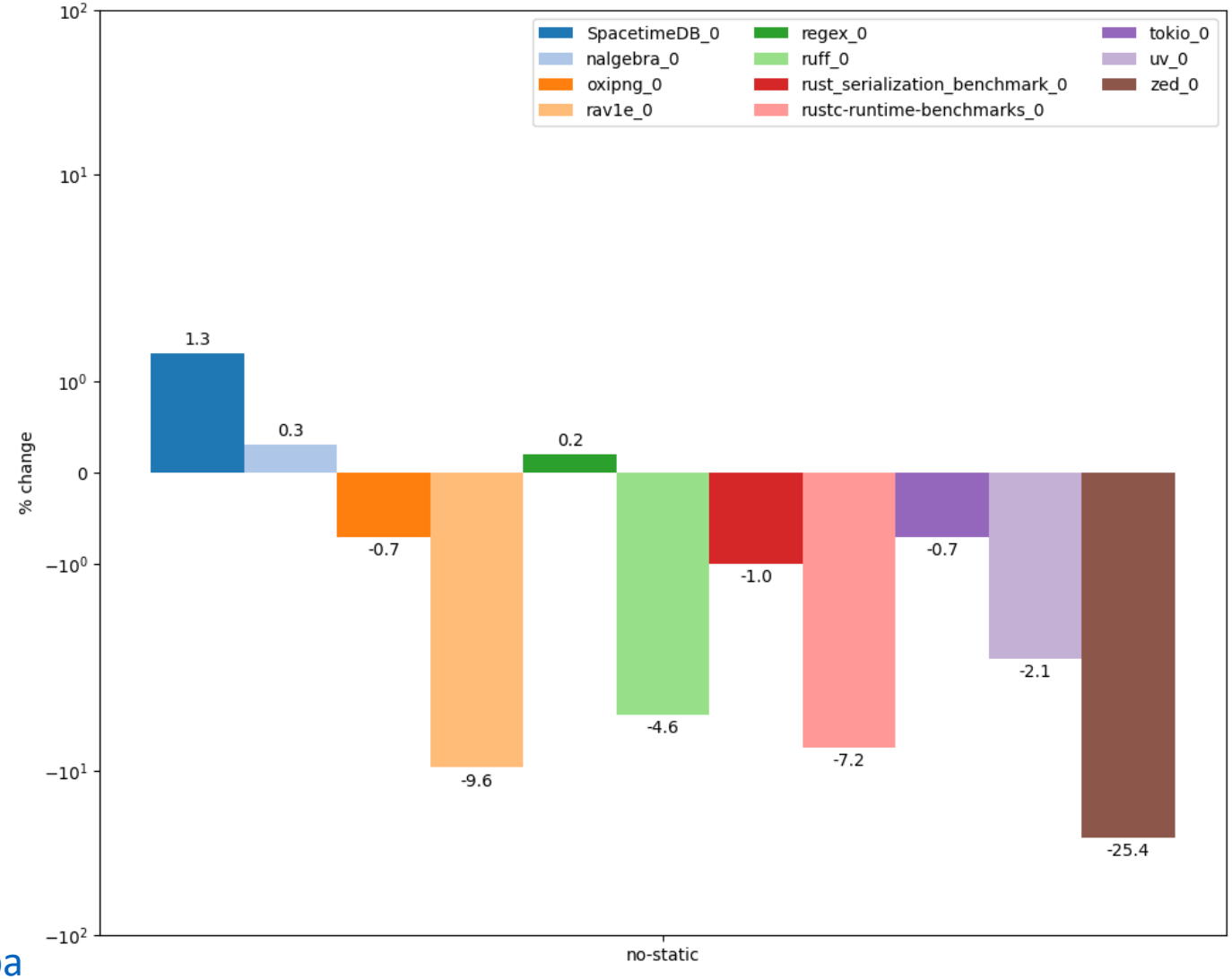
Сравнение с C/C++

- Многие авторитетные руководства рекомендуют локализовывать функции:
  - Google C++ Style Guide
  - LLVM Coding Standard (Restrict Visibility)
  - C++ Core Guidelines
  - Chromium C++ Style Guide
- Слабым местом C++ является невозможность локализовать private-методы классов
  - Попытка такой оптимизации

Распространённость

- Сколько локальных функций в реальных программах на Rust ?
  - 80-90%
  - (Для сбора статистики отключалась локализация с помощью патча)
- В C/C++ результаты сильно зависят от проекта:
  - 80-90% (clang, ffmpeg, git, opencv, bitcoin)
  - 40-50% (openssl, tmux, gcc, libuv)
  - (Для сбора статистики использовалась утилита Localizer)

Замедление при отключении локализации



Патч для компилятора

Выводы

- Локализация функций позволяет компилятору генерировать существенно более производительный код
- В Rust по умолчанию принято более эффективное (с точки зрения производительности) решение

Рекомендуемые материалы

- Efficient C Tips #5 – Make ‘local’ functions ‘static’

Отсутствие inplace-инициализации

Детальный разбор

Введение

- В C++ существуют механизм placement new для конструирования объектов в произвольном буфере в памяти:

```
void *addr = some_alloc(sizeof(A));
A *ptr = new(addr) A(...);  // Или std::construct_at
```

- А также perfect forwarding:

```
std::vector<LargeStruct> vec;
vec.emplace_back(large_struct_initializer);
```

- В языке Rust аналогичный механизм отсутствует
  - Следствие обязательной инициализации объектов
- Проблема:

```
// Сможет ли компилятор убрать копирования (со стека в кучу) ?
let b = Box::new(LargeStruct { ... });
```

Зачем нужны placement new ?

```rust
const SIZE: usize = 1024 * 1024 * 1024;

fn main() {
    let b: Box<[usize]> = if cfg!(fast) {
        vec![15; SIZE].into_boxed_slice()
    } else {
        // Проблемное копирование
        Box::new([15; SIZE])
    };
    black_box(&b);
}
```

```
# Сможет ли компилятор оптимизировать
копию ?
$ LIMIT=$((8 * 1024))
$ rustc +baseline -O test.rs
$ (ulimit -s $ LIMIT; ./test)
thread 'main' has overflowed its stack
fatal runtime error: stack overflow

# Работает с некопирующим dedicated API
$ rustc +baseline --cfg fast -O test.rs
$ (ulimit -s $ LIMIT; ./test)
```

Зачем нужны placement new ?

```
struct A {
  std::array<int, 1024> vals = {};

  A() = default;
  A(const A &) = default;

  A(int val) {
    std::fill(vals.begin(), vals.end(), val);
  }
};

int main() {
  std::vector<A> aa;
  aa.reserve(NITER);

  for (size_t i = 0; i < NITER; ++i) {
#ifdef USE_PLACEMENT_NEW
    aa.emplace_back(i);
#else
    A a(i);
    aa.push_back(a);
#endif

    asm("" :: "r"(&aa) : "memory");
  }

  return 0;
}
```

USE_PLACEMENT_NEW
быстрее на ~10% !

Оптимизации в компиляторе

- Move/copy elision
  - Обнаружение и удаление ненужных перемещений или копирований
- Может работать намного более агрессивно чем в C++
  - Наличие high-level IR (MIR)
  - Shallow move/copy (по сути move/copy в Rust это просто вызов memcpy)
- Компиляторы C++ не поддерживают полноценный copy elision
  - Работают ограниченные оптимизации типа RVO, NRVO, etc.
  - Прежде всего из-за отсутствия high-level IR (ClangIR to the rescue ?)
  - См. работу Романа Русяева по Ultimate Copy Elision (GitHub со ссылкой на реализацию)
- Реализация в компиляторе:
  - MIR (high-level IR): CopyProp, DeadStoreElimination, DestinationPropagation, RenameReturnPlace
  - LLVM: MemCpyOptPass

Детальный разбор

Проблема с move/copy elision

- Зависит от версии компилятора
  - Не всегда монотонно !
- Работает только в release-режиме
  - В debug-сборке копии остаются и могут вызвать переполнение стека или по крайней мере сильное замедление
- Языковая поддержка inplace-инициализации является одной из целей на 2026 год:
  - In-place initialization

Workarounds

- Те же что для обязательных инициализаций:
  - Использование специальных API для создания неинициализированных буферов и их явная инициализация
  - Например Box/Rc::new_uninit, Vec::with_capacity/set_len
- А также другие API например
  - Vec::into_boxed_slice
  - Vec::resize_with, Vec::copy_with, HashSet::get_or_insert_with, etc.
    - (Повышают шанс copy elision, но не гарантируют его)

Выводы

- Copy elision в Rust работает намного эффективнее чем в C++
- Но не может компенсировать отсутствие явной inplace-инициализации
  - Placement new и perfect forwarding
- Разработчики языка осознают эту проблему и активно работают над её решением:
  - Rust Project Goals 2026: In-place initialization
  - Rust Project Goals 2026: MIR move elimination

Рекомендуемые материалы

- TODO

Оптимизации расположения полей

Детальный разбор

Введение

- TODO

Пример

- TODO

Накладные расходы

- TODO

Оптимизации в компиляторе

- TODO

Workarounds

- TODO

Выводы

- TODO

Рекомендуемые материалы

- TODO

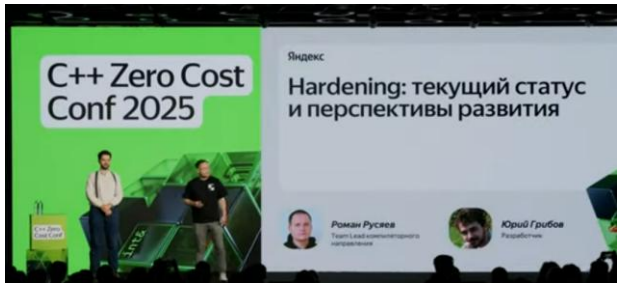
Арифметические проверки

Детальный разбор

Введение

- Переполнение арифметических операций может приводить к серьёзным системным сбоям
  - ~1% CVE и 1.5% KEV в 2024
  - 23 место в рейтинге Mitre CWE Top 25 2024 (8 в рейтинге 2019 года)
- Наиболее известные примеры:
  - Инцидент с облучателем Therac-25 (1985)
  - Катастрофа ракеты Ariane 5 (1996)
- История (pre-Rust):
  - `-ftrapv` появилась в GCC в 2000 (patch for -ftrapv option)
    - За фичей не следили и она быстро протухла (например BZ #35412 открыт в 2008)
  - Работы John Regehr в 2010 (IOC)
  - Создание UBsan в 2014 (на волне популярности Asan)
    - State-of-the-art решение
- Дополнительная информация:
  - Hardening: текущий статус и перспективы развития

J.R. Armstrong, Famous Failures Revisited



Пример ошибки (OpenSSH 3.3)

```
nresp = packet_get_int();
if (nresp > 0) {
  // Переполняем целое число до нуля здесь ...
  response = xmalloc(nresp*sizeof(char*));
  // ... и вызываем heap buffer overflow тут
  for (i = 0; i < nresp; i++)
    response[i] = packet_get_string(NULL);
}
```

Дефолтные арифметические проверки Rust

- Некоторые (не самые опасные...) арифметические проверки включены по умолчанию
- Деление на ноль
- Знаковое переполнение при делении (INT_MIN / -1)
- Ограничение диапазона при преобразовании float в integer
  - Можно отключить с помощью -Zsaturating-float-casts=off
- Целочисленные переполнения в контейнерах стандартной библиотеки
  - С помощью assert!, checked_add, etc.

Важные проверки, отсутствующие в Rust

- По историческим причинам некоторые важные проверки отсутствуют
  - И даже не могут быть включены по опции
- Например переполнение при преобразовании типа с помощью as
  - Нужно пользоваться альтернативным API try_into/ try_from

```rust
use std::convert::TryFrom;

fn main() {
    if cfg!(try_from) {
        i8::try_from(0x100i32).unwrap();
    } else {
        0x100i32 as i8;
    }
}
```

```
$ rustc +baseline test.rs -O
$ ./test
0
$ rustc +baseline --cfg try_from test.rs -O
$ ./test

thread 'main' panicked at tmp57.rs:5:32:
called `Result::unwrap()` on an `Err` value:
TryFromIntError(())
```

Проверки переполнений в Rust

- По умолчанию целочисленные переполнения в пользовательском коде проверяются *только* в debug-сборках
- Two’s complement в release
- Можно включить проверки по опции -C overflow-checks=on
- Причины: слишком большие накладные расходы
  - “Plan is to turn on checking by default *if* we can get the performance hit low enough and also that it should leave room in the future to move towards universal overflow checking if it becomes feasible” (Niko Matsakis, ко-lead Rust language design team)

Defined overflow

The Value of Undefined Behavior и Garbage In, Garbage Out

- В отличие от C/C++ целочисленное переполнение не является Undefined Behavior
  - Паника или Two’s complement
- Это лишает оптимизатор дополнительной информации
- Может помешать точности анализов (особенно ScalarEvolution) и оптимизаций в LLVM
  - См. также How undefined signed overflow enables optimizations in GCC

```
// Для знаковых i1/i2 компилятор
// может оптимизировать
// индуктивные переменные
// для &buf[i1] и &buf[i2].
// Если беззнаковых нет.

T i1 = 0, i2 = 0;

for (;;) {
    int c1 = buf[i1];
    int c2 = buf[i2];
    if (c1 != c2)
        return c1 - c2;
    i1++;
    i2++;
}
```

Проблема с inclusive ranges

- Циклы с включенной верхней границей оптимизируются хуже чем обычные:

```
for i in 1..=n {
    total += i;
}
```

- Из-за возможного переполнения (при `n==T::MAX`) компилятор выполняет их с дополнительной проверкой:

```
while i <= n {
    total += i;
    if i >= n {
        break;
    }
    i += 1;
}
```

- Можно помочь компилятору с помощью явного вычисления границы:

```
for i in 1..(n+1)
```

- Примеры: Why isn't the for loop optimized better?, Comparing Pythagorean triples in C++, D, and Rust, etc.

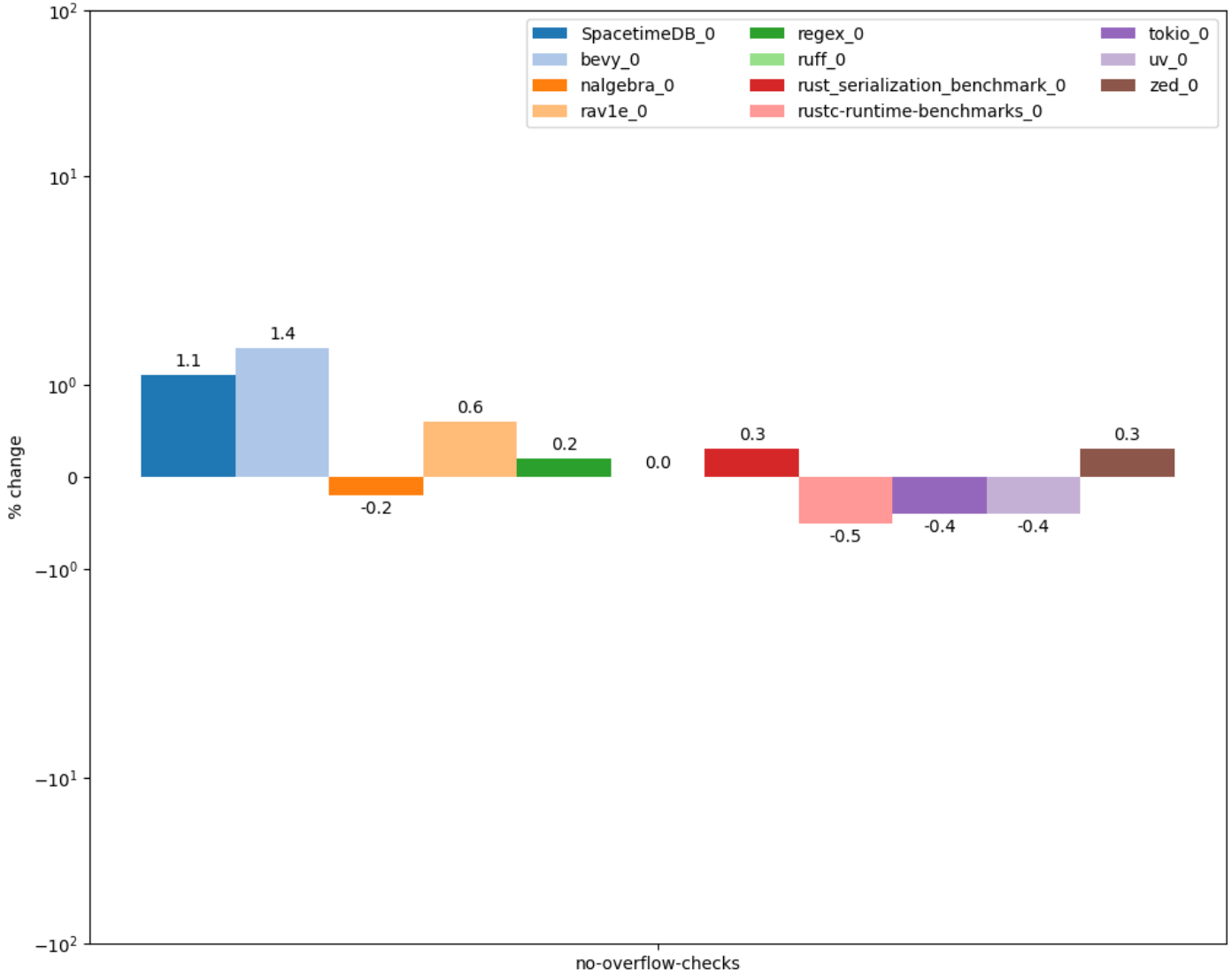
Другие накладные расходы

- Накладные расходы от проверок:
  - Сравнение + предсказуемый условный переход (2 такта конвеера + I$/BTB pressure)
  - Мешают другим оптимизациям (vectorizer, etc.)
- С первым (менее важным) пунктом *возможно* могла бы помочь аппаратная поддержка проверок в процессоре

```rust
#[no_mangle]
pub fn foo(xs: &[i32]) -> i32 {
    let mut ans = 0;
    for x in xs {
        ans += x;
    }
    ans
}
```

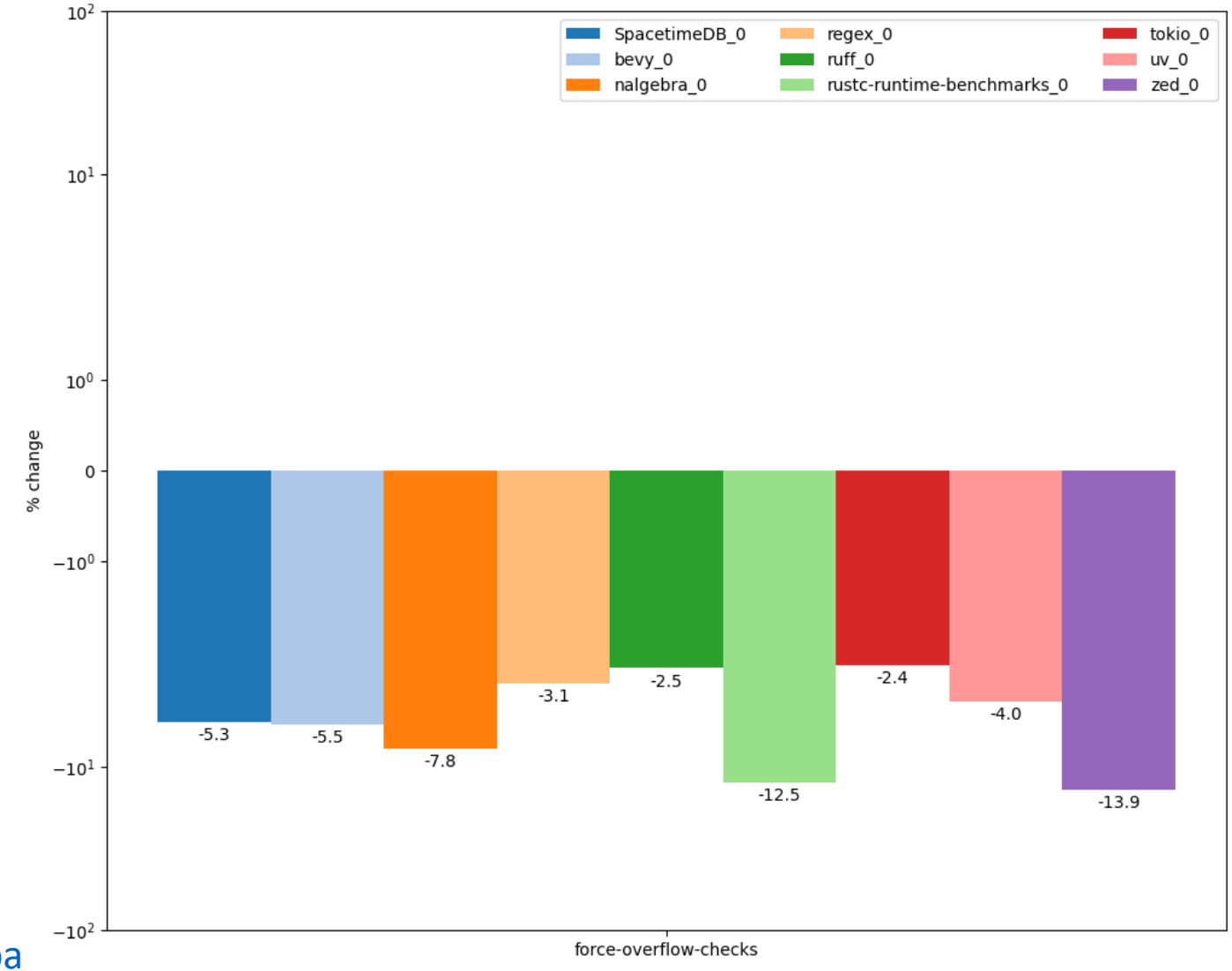
```
$ rustc ... test.rs
...
.LBB0_5:
        // Цикл успешно векторизуется
        movdqu  xmm2, xmmword ptr [rdi + 4*rax]
        paddd   xmm0, xmm2
        ...
$ rustc ... -Coverflow-checks=on test.rs
...
.LBB0_3:
        // Цикл не векторизуется
        add     eax, dword ptr [rdi + rcx]
        jo      .LBB0_6
        add     rcx, 4
        cmp     rsi, rcx
        jne     .LBB0_3
        ...
```

Улучшение при отключении дефолтных проверок (деление на ноль и т.д.)



Ветка компилятора

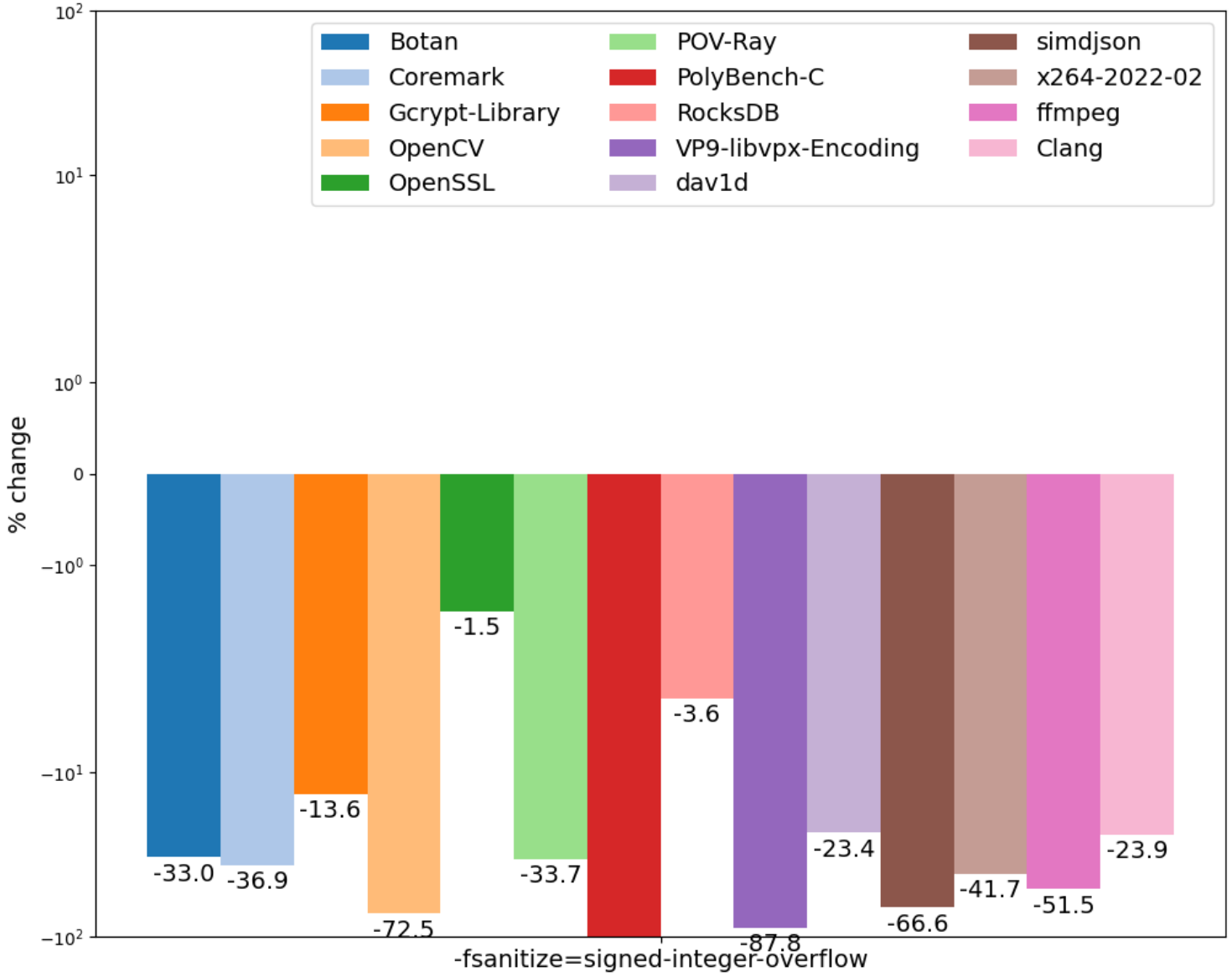
Замедление при включении целочисленных переполнений



- Существенный рост числа проверок (+30% проверок в rustc)

Ветка компилятора

Замедление в C++



- Аналогичный или даже *больший* оверхед в UBsan для C/C++

Оптимизации в компиляторе

- Проверки переполнений намного хуже оптимизируются компилятором
  - Арифметика Пресбургера разрешима, но имеет сложность O(2^2^N)
- При сборке stage 2 rustc компилятор оптимизирует ~30% арифметических проверок
  - Данные получены на основе анализа бинарного кода

Workarounds

- Небезопасные API для арифметики:
  - unchecked_add, wrapping_add, etc.
  - to_int_unchecked
  - core::hint::unreachable_unchecked

```
if x <= i32::MIN >> 1 || x >= i32::MAX >> 1 {
    unsafe { std::hint::unreachable_unchecked() }
}
x * 2 / 2
```

  - (проверки в стандартной библиотеке *не могут* быть отключены)
- Специальный типы стандартной библиотеки:
  - NonZeroU32 для отключения некоторых проверок деления
  - Wrapping<T> для переполнений

Выводы

- В Rust целочисленное переполнение не является UB
  - Мешает компилятору проводить некоторые агрессивные оптимизации
- В языке приняты неочевидные решения о дефолтных арифметических проверках
  - Могут давать замедление ~1-2%
- Целочисленные переполнения имеют слишком большие накладные расходы (до 10%)
  - Аналогично UBsan
  - Вряд ли будут включены по умолчанию в будущем

Рекомендуемые материалы

- RFC 560: Integer Overflow (а также её обсуждение и обзор)
  - Попытка включить проверки переполнений в release
- Understanding Integer Overflow in C/C++
  - Статья John Regehr
- How undefined signed overflow enables optimizations in GCC
  - Статья о пользе Undefined Behavior для integer overflow в C/C++

Стандартная библиотека

Детальный разбор

Введение

- По сравнению с STL в стандартной библиотеке Rust делается больше обязательных проверок и используются более устойчивые (и медленные) алгоритмы
- Но в некоторых ситуациях могут использоваться более быстрые контейнеры и алгоритмы

Проверки в стандартной библиотеке

- Стандартная библиотека Rust содержит большое количество неотключаемых проверок (assert!, checked_add, etc.)
- Основные виды проверок:
  - Корректность индексов в контейнерах (bounds checks, уже рассматривались выше)
  - Отсутствие переполнений при аллокациях (capacity overflow)
  - Успешность аллокаций памяти в контейнерах
  - Корректные обращения к UTF-8 контейнерам &str и String
  - API-специфичные проверки (аксиомы компараторов, etc.)
- Некоторые аналогичные проверки доступны и в STL
  - По макросам _GLIBCXX_ASSERTIONS и _LIBCPP_HARDENING_MODE

Пример

```rust
pub fn foo(length: usize) -> Vec<i32> {
    vec![-1; length]
}
```

```
        lea     r15, [4*rsi]
        xor     r13d, r13d
        mov     rax, rsi
        // Проверяем что 4*length не переполняется
        shr     rax, 62
        jne     .LBB0_11
        // Проверяем что 4*length <= isize::max
        movabs  rax, 9223372036854775804
        cmp     r15, rax
        ja      .LBB0_11
        ...
        // Проверяем успешность аллокации
        call    qword ptr [rip +
_RNvCslKhMTRUAIjP_7___rustc12___rust_alloc@GOTPCREL]
        test    rax, rax
        je      .LBB0_11
```

Пример

```rust
use std::io;

fn main() {
    let mut input = String::new();
    io::stdin()
        .read_line(&mut input)
        .unwrap();
    println!("{}", input);
}
```

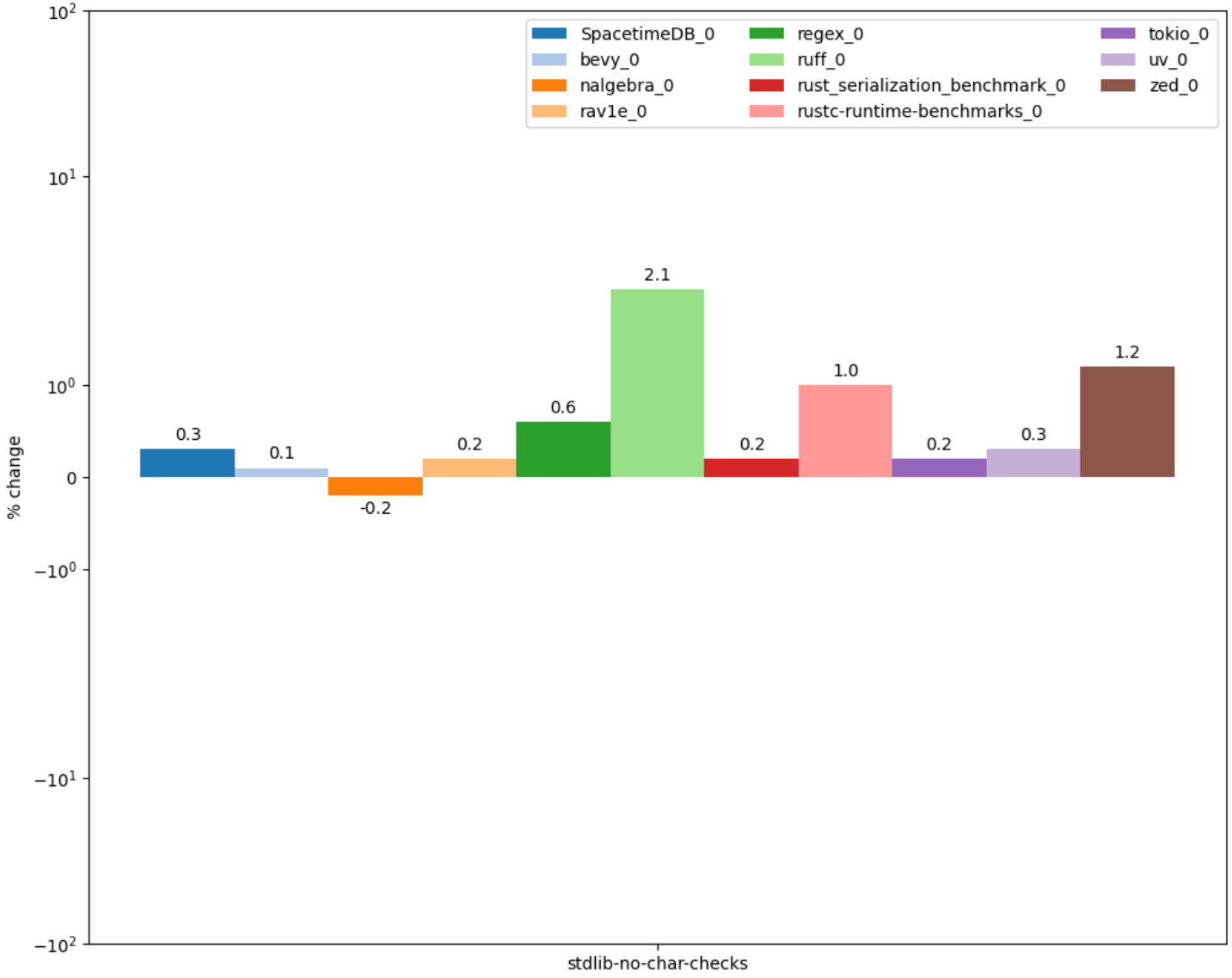
```
$ echo Привет | ./test
Привет

$ echo Привет | iconv -f UTF-8 -t KOI8-RU | ./test
thread 'main' panicked at test.rs:5:39:
called `Result::unwrap()` on an `Err` value:
Error { kind: InvalidData, message: "stream did not contain valid UTF-8" }
```

Другие накладные расходы

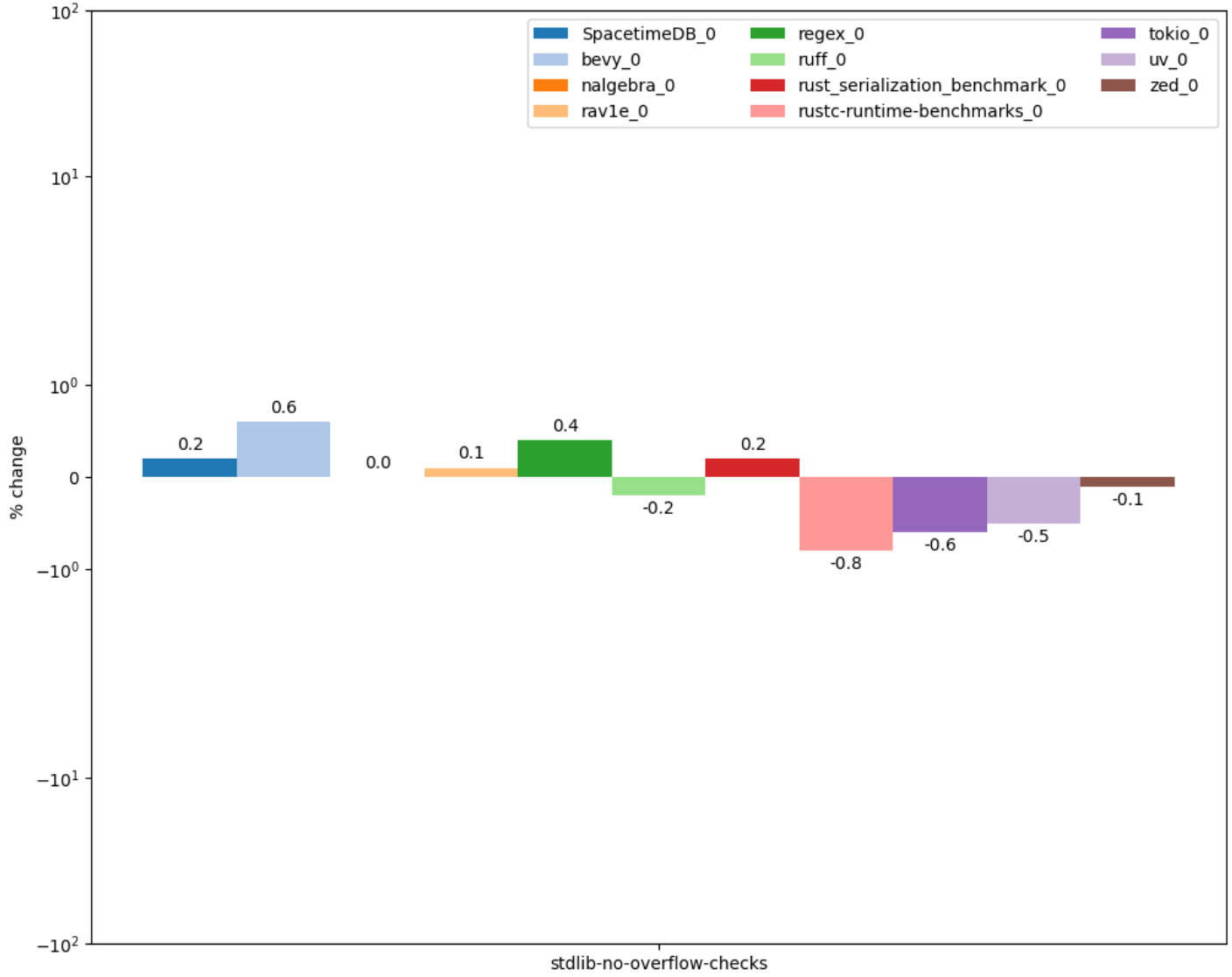
- Генератор псевдослучайных чисел в Rust является криптографически безопасным и может быть более медленным чем rand(3) или std::mt19937
- Дефолтный алгоритм хэширования (SipHash) устойчив к DoS-атакам, но является более медленным чем аналогичные алгоритмы в STL
- Ввод-вывод в Rust по умолчанию не буферизуется
  - Без использования BufReader программы могут работать в десятки раз медленней
- io::stdout() строчно-буферизуем даже для не-TTY вывода (rust #60673)
  - Это может сильно замедлять command-line утилиты (в ситуциях типа “./prog > file”)
- Макросы print!/println! используют монопольный доступ для обеспечения корректного выводы из параллельных потоков

Улучшение при отключении проверок UTF-8



Ветка компилятора

Улучшение при отключении проверок переполнений в контейнерах



Ветка компилятора

Оптимизации в стандартной библиотеке

- Хэштаблицы:
  - Нет требования валидности ссылок после изменения таблицы (как в std::unordered_map)
  - Это позволяет использовать open-addressing вместо chaining (меньше аллокаций, cache-friendly)
- В деревьях поиска используется B-tree (вместо RB-tree как в std::map)
  - Меньше аллокаций, cache-friendly
  - 4x ускорение на простом микробенчмарке
- Современные ILP-friendly алгоритмы сортировки (ipnsort, driftsort)
  - STL-алгоритмы написаны для упрощения векторизации при сортировке примитивных типов
  - 65% ускорение на простом микробенчмарке

Выводы

- TODO

Рекомендуемые материалы

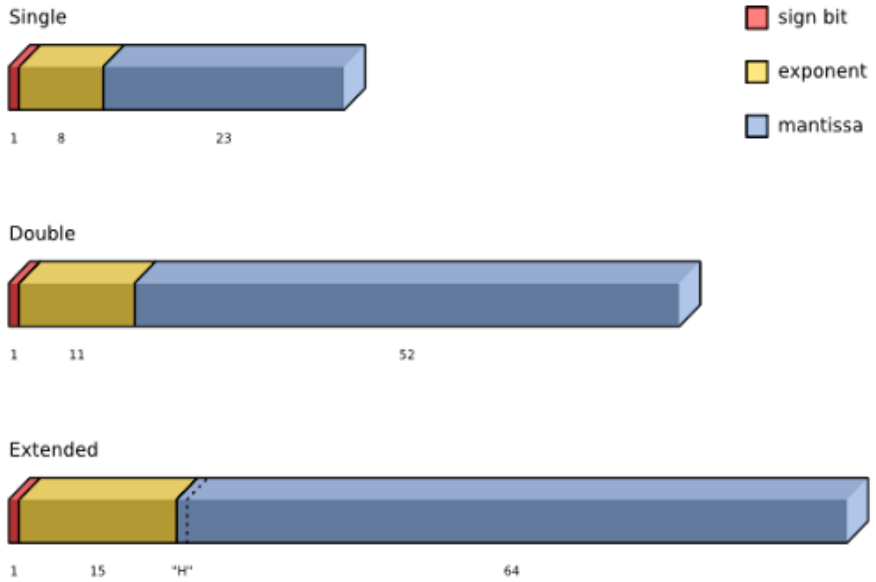
- Safety vs Performance. A case study of C, C++ and Rust sort implementations

Отсутствие режима fast-math

Детальный разбор

Вычисления с плавающей запятой

- Стандарт IEEE 754
  - Типы и их представление
  - Операции над ними
  - Округление
- Большинство архитектур, языков и компиляторов следуют стандарту (почти полностью)
- Причины расхождений
  - Аппаратная специфика
  - Больше возможностей для оптимизации
- fast-math – собирательный термин для оптимизаций, связанных с ослаблением требований IEEE 754



Виды fast-math флагов

- Алгебраические:
  - Ассоциативность
  - Беззнаковость нуля
  - Замена деления на умножение
  - Объединение операций (например FMA)
- Запрет нестандартных значений:
  - NaN, Infinity

```
double foo(double val) {
  return (val + 3.0) / 3.0;
}
```

```
.LCPI0_0:
        .quad   0x4008000000000000 # 3.0
foo:
        movsd   xmm1, qword ptr [rip +
.LCPI0_0]
        addsd   xmm0, xmm1
        divsd   xmm0, xmm1
        ret
```

clang-21 -O2

```
.LCPI0_0:
        .quad   0x4008000000000000 # 3.0
.LCPI0_1:
        .quad   0x3fd5555555555555 # 1/3
foo:
        addsd   xmm0, qword ptr [rip +
.LCPI0_0]
        mulsd   xmm0, qword ptr [rip +
.LCPI0_1]
        ret
```

clang-21 -O2 -freciprocal-math

Как на это смотрят языки программирования

- У разных языков различные отношения к соблюдению IEEE 754
- Стандарты C и C++ запрещают fast-math оптимизации
  - Но обычно их можно включить с помощью флагов (например -ffast-math)
  - Часто используется в игровых движках
- Аналогичное поведение для языка Swift
- Java и C# разрешают совершать вычисления с большей точностью, чем результирующий тип
- Go разрешает объединение операций
  - Но только если этому не препятствуют явные преобразования типов
- Fortran явно разрешает нарушать IEEE 754 ради производительности

Оптимизации: Векторизация

- -fassociative-math и -fno-signed-zeros позволяет компилятору переупорядочить итерации цикла и векторизовать вычисления

```c
#include <math.h>

float sum256(float *arr1, float *arr2) {
  float s;
  int i;
  s = 0.0f;
  for (i = 0; i < 256; i++) {
    s += arr1[i] * arr2[i];
  }
  return s;
}
```

```asm
.LBB0_1:
  movss   xmm1, dword ptr [rdi + 4*rax]
  mulss   xmm1, dword ptr [rsi + 4*rax]
  addss   xmm0, xmm1
  # один элемент за итерацию цикла
  inc     rax
  cmp     rax, 256
  jne     .LBB0_1
```

clang-21 -O2 -fno-unroll-loops

```asm
.LBB0_1:
  movups  xmm1, xmmword ptr [rdi + 4*rax]
  movups  xmm2, xmmword ptr [rsi + 4*rax]
  mulps   xmm2, xmm1
  addps   xmm0, xmm2
  # 4 элемента за итерацию цикла
  add     rax, 4
  cmp     rax, 256
  jne     .LBB0_1
```

clang-21 -O2 -fno-unroll-loops
-fassociative-math -fno-signed-zeros

Оптимизации: Упрощение выражений

- Fast-math флаги расширяют возможности компилятора по вычислению выражений во время компиляции
- -fassociative-math + -fno-signed-zeros

```
double foo(double val) {
  return 3.0 + val - 3.0;
}
```

```
foo:
  addsd   xmm0, qword ptr [rip + .LCPI0_0] # +3.0
  addsd   xmm0, qword ptr [rip + .LCPI0_1] # -3.0
  ret
```

clang-21 -O2

```
foo:
  ret
```

clang-21 -O2
-fassociative-math -fno-signed-zeros

- -fassociative-math + -fno-signed-zeros + -freciprocal-math

```
double foo(double val) {
  return 3.0 * val / 3.0;
}
```

```
foo:
  movsd   xmm1, qword ptr [rip + .LCPI0_0]
  mulsd   xmm0, xmm1 # * 3.0
  divsd   xmm0, xmm1 # / 3.0
  ret
```

clang-21 -O2

```
foo:
  ret
```

clang-21 -O2
-fassociative-math -fno-signed-zeros
-freciprocal-math

Оптимизации: Сложные инструкции

- Некоторые архитектуры поддерживают более сложные операции над числами с плавающей точкой
  - Например, Fused Multiply Add в x86-64
- Алгоритм округления в этих инструкция может отличаться от последовательного выполнения элементарных операций
- Флаг -ffpcontract=on позволяет компилятору использовать эти инструкции

```
double foo(double *val) {
  double acc = 1;
  for (int i = 0; i < 100; ++i) {
    acc = acc * val[i] + val[i + 1];
  }
  return acc;
}
```

```
.LBB0_1:
        mulsd   xmm0, xmm1 # multiply
        movsd   xmm1, qword ptr [rdi + 8*rax]
        addsd   xmm0, xmm1 # add
        inc     rax
        cmp     rax, 101
        jne     .LBB0_1
        ret
```

clang-21 -O2 -fno-unroll-loops

```
.LBB0_1:
        vmovsd  xmm2, qword ptr [rdi + 8*rax]
        vfmadd213sd     xmm0, xmm1, xmm2 # fused multiply-add
        inc     rax
        vmovapd xmm1, xmm2
        cmp     rax, 101
        jne     .LBB0_1
        ret
```

clang-21 -O2 -fno-unroll-loops -ffp-contract=on -march=haswell

Оптимизации: Finite only

- -ffinite-math-only разрешает оптимизации, основанные на том, что операнды не принимают NaN или Inf значений
- Это позволяет:
  - Удалять проверки на NaN и Inf
  - x == x => true
  - x / x => 1
- А с -fno-signed-zeros:
  - x – x => 0
  - 0 / x => 0
  - x * 0 => 0

```
bool foo(double val) {
  return val == val;
}
```

```
foo:
  ucomisd xmm0, xmm0
  setnp   al
  ret
```

clang-21 -O2

```
foo:
  mov     al, 1
  ret
```

clang-21 -O2
-ffinite-math-only

```
double foo(double val) {
  return val * 0;
}
```

```
foo:
  xorpd   xmm1, xmm1
  mulsd   xmm0, xmm1
  ret
```

clang-21 -O2

```
foo(double):
  xorps   xmm0, xmm0
  ret
```

clang-21 -O2
-ffinite-math-only
-fno-signed-zeros

Подводные камни: Проверки

- Появление Inf или NaN значений при использовании -ffinite-math-only приводит к UB
- Если отсутствие этих значений обеспечивалось явными проверками, то они будут удалены компилятором

```
#include <math.h>

int foo(float f) {
  return isnan(f);
}
```

```
foo:
  xor     eax, eax
  ucomiss xmm0, xmm0
  setp    al
  ret
```

clang-21 -O2

```
foo:
  xor     eax, eax
  ret
```

clang-21 -O2
-ffinite-math-only

Подводные камни: Округление

- Некоторые алгоритмы используют специфику IEEE 754 и специальный порядок вычислений для уменьшения погрешности округления
- С -fassociative-math, компилятор “оптимизирует” эти алгоритмы так, как будто погрешности округления нет
- Алгоритм Кэхэна

```c
float sum(float *arr) {
  float s;
  int i;
  s = 0.0f;
  for (i = 0; i < 256; i++) {
    s = s + arr[i];
  }
  return s;
}
```

```asm
sum_kahan:
  lea     rax, [rdi+1024]
  pxor    xmm0, xmm0
.L2:
  addss   xmm0, DWORD PTR [rdi]
  add     rdi, 4
  cmp     rdi, rax
  jne     .L2
  ret
```

clang-21 -O2 -ffast-math

```c
float sum_kahan(float *arr) {
  float s, c, t, y;
  int i;
  s = c = 0.0f;
  for (i = 0; i < 256; i++) {
    y = arr[i] - c;
    t = s + y;
    c = (t - s) - y;
    s = t;
  }
  return s;
}
```

Позиция Rust

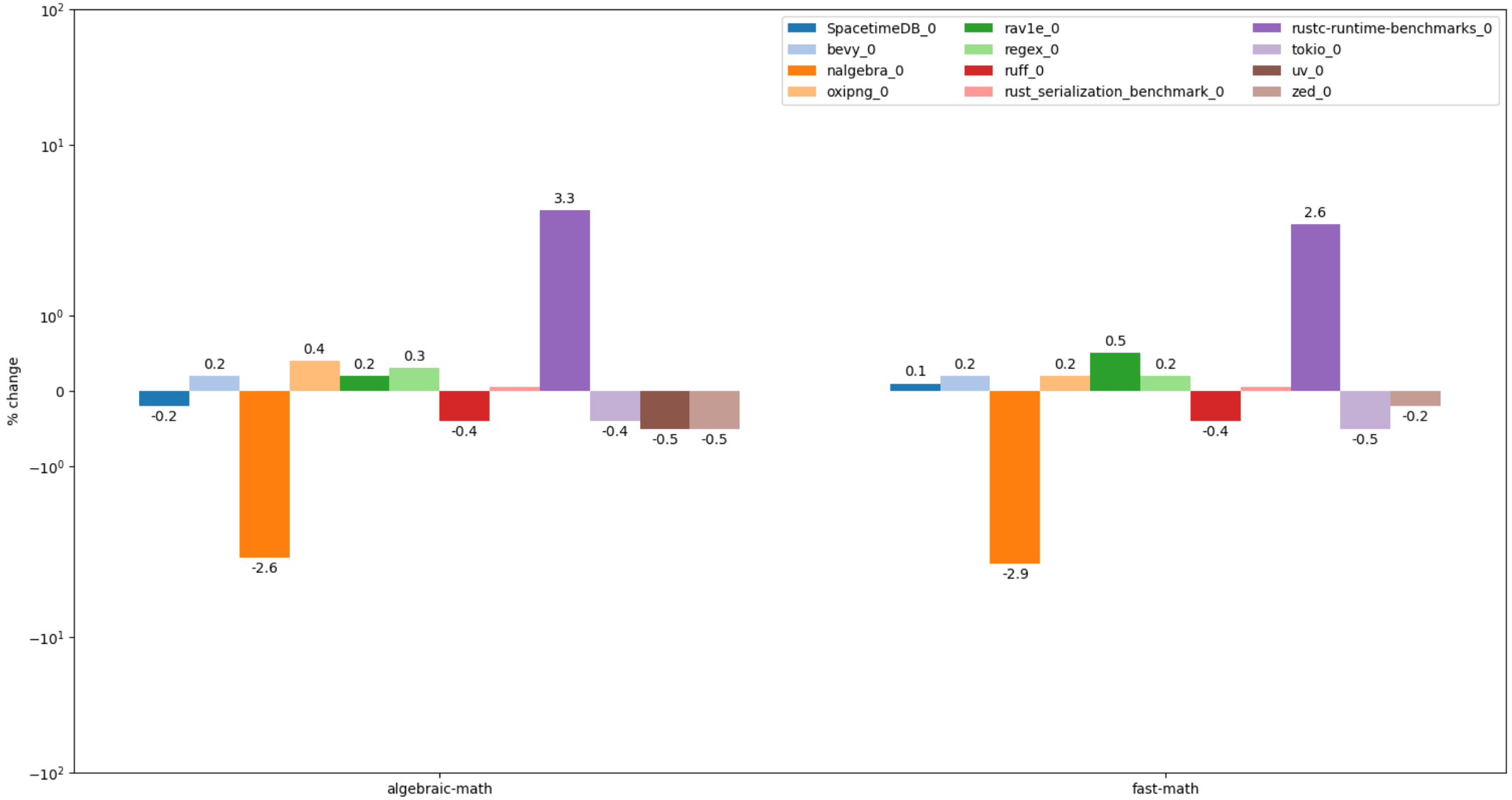
- Rust отказывается от возможности глобального применения fast-math оптимизаций
- Основная мотивация – опасность недостаточно осторожного применения этих оптимизаций, особенно в зависимостях проекта

“Everything about this flag fundamentally clashes with the idea of robust, compositional system design. It was a terrible mistake to ever add it to C compilers, and Rust should not repeat that mistake. Rust should instead explore alternative, less fragile ways of exposing those semantics.” (Ralf Yung, rust #21690)

Workarounds: Интринсики

- Rust предоставляет fast-math оптимизации на уровне отдельных операций
  - std::intrinsics::f{op}_fast
  - std::intrinsics::f{op}_algebraic
  - f(16/32/64)::algebraic_{op}
- На данный момент эти API не доступны в стабильной версии

Результаты при включении fast-math



[Ветка компилятора](#) [Ветка компилятора](#)

Выводы

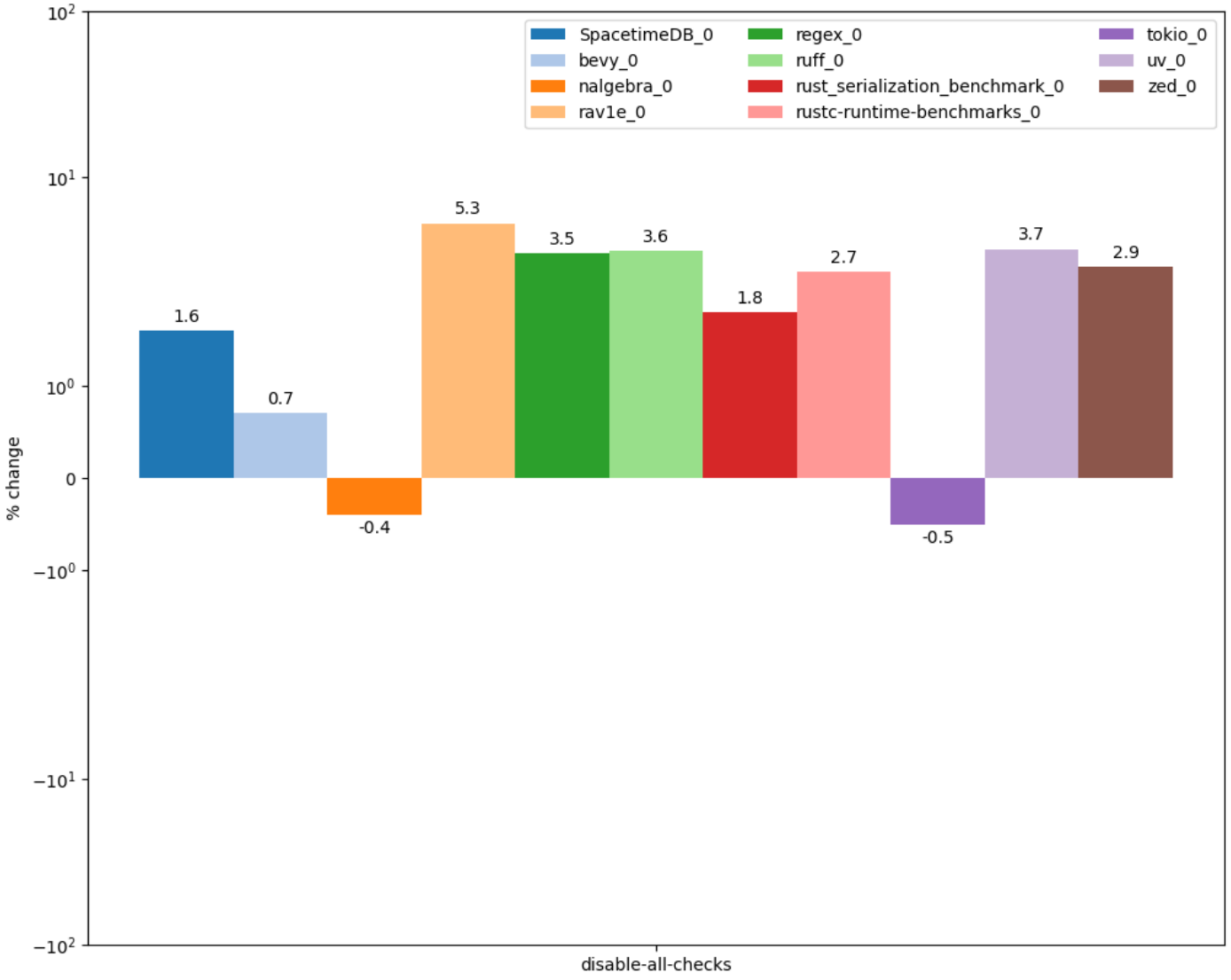
- TODO

Рекомендуемые материалы

- TODO

Выводы

Отключение (почти) всех проверок Rust



- Отключались проверки индексов, переполнения контейнеров stdlib, проверки UTF-8 в строках
- Остаются накладные расходы на избыточную инициализацию (полагаем не менее 1%)

Ветка компилятора

Выводы

- Сообщество Rust следит за балансом безопасности и производительности:
  - Бескомпромиссно включаются только проверки memory safety
  - Остальные делаются только если не влияют существенно на производительность
  - Например по умолчанию отключены проверки integer overflow
- Оверхед динамических проверок зависит от проекта и может составлять 3-6%
  - TODO: сравнить с Hardened C/C++ ?
- Факторы, наиболее существенно влияющие на производительность программ:
  - Индексные проверки, автоинициализация, отсутствие inplace-инициализации
- Скорее всего будут оптимизироваться в будущих версиях Rust и LLVM
- Но полностью их негативное влияние обойти нельзя (по крайней мере в текущей версии языка)
- Всегда будут требовать тюнинга и неидиоматичного кода для достижения оптимальной производительности

О чём мы не успели рассказать ...

- Codegen units
- Особенности реализации паник (исключений)
  - В целом накладные расходы аналогичны исключениям в C++
- GC-sections by default
- Особенности ABI
- Отключение PLT
- Особенности реализации виртуальных методов
- Проверки Stack Clashing

Вопросы ?

Последняя версия слайдов

Подготовка доклада

- Собрано и проанализировано более 350 статей, блогпостов и обсуждений по теме доклада из разных источников (materials)
- Общие временные затраты на подготовку (на февраль 2026):
  - > 2.5 человекомесяцев
- Анализ проводился для
  - Rust 1.87.0 (май 2025)
  - Clang llvmorg-20.1.7 (июнь 2025)

Rust-бенчмарки

- Внутренние бенчмарки крупных open-source проектов из разных областей:
  - Gamedev (Bevy, Veloren)
  - Базы данных (SpacetimeDB)
  - Работа с текстом (Zed, Ruff, regex)
  - Кодеки (rav1e, oxipng)
  - Линейная алгебра (nalgebra)
  - Асинхронный runtime (tokio)
  - Пакетные менеджеры (uv)

Rust-бенчмарки

- Сервера настраивались в соответствии с методологией uInit
- Для сравнения использовались стандартные Criterion-бенчмарки каждого проекта (cargo bench), усреднённые с помощью geomean
  - Некоторые особо медленные тесты в oxipng были отключены
  - Для борьбы с системными шумами бралось минимальное значение средних из нескольких прогонов
  - Важно: 1% geomean может означать что
    - Все тесты в проекте замедлились на 1%
    - 10% тестов замедлилось на 10%
  - Отдельные тесты могли замедлиться (или ускориться) на десятки процентов
- Дополнительные детали здесь

C++ бенчмарки

- Сервера настраивались в соответствии с методологией uInit
- Использовался компилятор Clang версии llvmorg-20.1.7
- Llvm-bench (benchmarks/cpp/llvm-bench)
  - Замерялось медианное время O2-компиляции файла CGBuiltin.cpp (самый большой файл в LLVM) компилятором Clang
  - Тестируемый Clang был собран самим собой
- Ffmpeg-bench (benchmarks/cpp/ffmpeg-bench)
  - По мотивам Stack Smashing Protection and Its Performance Impact
  - Замерялось медианное время конвертации файла FLV в MP4
- Phoronix Test Suite (benchmarks/cpp/PTS)
  - Популярный набор бенчмарков с phoronix-test-suite.com
  - Замерялись бенчмарки botan, bullet, coremark, c-ray, dav1d, gcrypt, gmpbench, luajit, nginx, opencv, openssl, polybench-c, povray, rocksdb, simdjson, vpxenc, x265 (см. #55)
  - Использовалось медианное время работы бенчмарка
  - Игнорировались тесты с Stdev > 0.5% и изменениями < 1%
- Для борьбы с системными шумами бралось минимальное значение медиан из 2-3 прогонов

Тестовый сервер

- Большинство (но не все) бенчмарков были замерены на Intel Xeon E-2236
  - С частотой зафиксированной на 2 ГГц
  - С установленной Ubuntu 24.04

TODO

- Обновить инфу по замерам (в analysis.md и презентации)
- График с включением всех проверок в Hardened C++
- Перемерять Rust no-static